



软件源代码缺陷检测

taos-conconnector-odbc

检测报告

苏州棱镜七彩信息科技有限公司

2026-04-17 21:38:30



2004

2010

2012

2006

CHART SEGMENT TITLE

CHART SEGMENT TITLE

报告声明

棱镜七彩是苏州棱镜七彩信息科技有限公司的简称，PrismSAST 是棱镜七彩具有自主知识产权的软件源代码安全缺陷检测平台的简称。

本检测报告由棱镜七彩软件源代码安全缺陷检测平台（PrismSAST）采用智能检测引擎进行自动化检测，并对检测结果经过人工复核后产生的检测报告。棱镜七彩保留对检测报告中各款项的解读权利，但并不意味着对检测结果中的缺陷或安全漏洞可能引起的任何直接经济损失或任何间接损失承担责任。

本检测报告的最终解释权归属棱镜七彩。

1 项目信息

项目名称	taos-connector-odbc
提交时间	2026-04-20 17:03:52
完成时间	2026-04-20 17:04:25
用时	00:00:33
代码总行数	64695
检测文件数	103
缺陷总数	456
致命缺陷数	0
缺陷密度（缺陷/KLOC）	7.05
检测类别	常规检测
报告创建日期时间	2026-04-17 21:38:30
检测团队	安全部
版本号	1.3.0

参考标准

*CMMI（软件能力成熟度模型集成）中缺陷密度参考值：		
CMM5 级	< 0.32	国际一流软件研发企业
CMM4 级	< 0.92	
CMM3 级	< 2.39	国内一流软件研发企业
CMM2 级	< 5.52	国内其他软件研发企业
CMM1 级	< 11.95	

2 风险等级

缺陷/安全漏洞严重等级	等级描述
致命	会造成程序崩溃或系统性故障的缺陷；或可能被容易利用且造成系统重大风险的安全漏洞
严重	会影响程序部分功能和性能，或者造成未定义行为的缺陷；或可能被利用且造成较为严重的风险的安全漏洞
一般	会造成对程序功能或性能的轻度影响；或存在可能被利用的低危安全漏洞。
强制	强制性标准是要求必须严格执行的标准条款
建议	建议性标准是企业可参考，自愿采用的标准条款
提示	不会直接影响程序的功能和性能，但这不是良好的代码风格，可能会有潜在的影响。
此等级划分是按照相关行业标准、国家标准，并参考《GB/T30279-2020 信息安全技术网络安全漏洞分类分级指南》进行定义	

3 评分评级

1. 用户可以在项目级别自定义下列三个影响因子的权重及情况：

序号	影响因子	描述
A	是否外部可达	可以分为物理隔绝、内部局域网连接、连接外网三个级别
B	影响程度	可以分为对别的组件或是部门影响较大、有一般的影响、不影响其它组件或是部门
C	缺陷/安全漏洞严重等级	致命、严重、一般、强制、建议、提示

2. 评分计算公式：A*A 权重 + B*B 权重 + C*C 权重

A、B、C 取值在【0，9】，三个权重相加等于 1，因此结果取值范围也在【0，9】；

最后根据公式的结果分级：低危【0，4），中危【4，6），高危【7，9）。

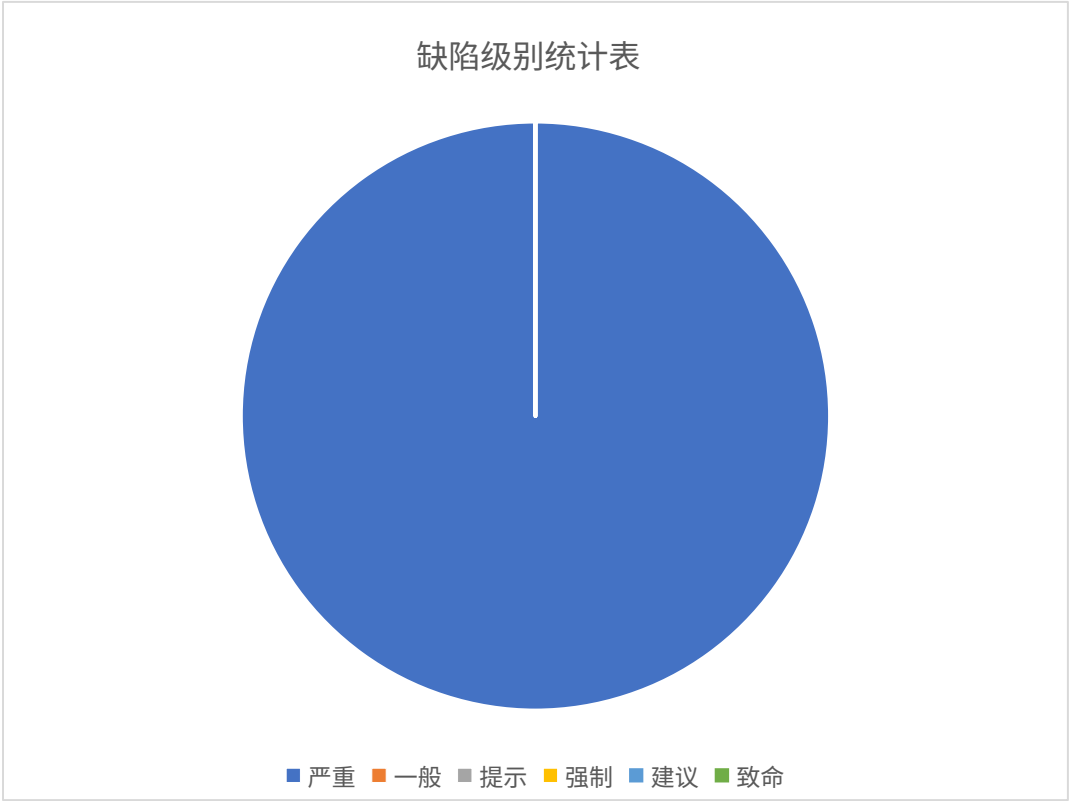
3. 缺省情况下：A 权重 20%，B 权重 20%，C 权重 60%

A = 5，B=5，C：致命=9，严重=7，一般=3，提示=1。

4 汇总统计

4.1 图表统计

4.1.1 按缺陷级别统计



4.1.2 按 CWE 2023 (Top25)统计



4.1.3 按 OWASP 2021 (TOP10)统计



4.2 选择检测集及检测结果统计表

序号	规则集	检测集	选择数量	发现缺陷/安全漏洞数
1	C/C++	CWE C/C++	24	0
2	C/C++缺陷类型	数据加密	3	0
3	C/C++缺陷类型	初始化问题	14	63
4	C/C++缺陷类型	缓冲区溢出	5	0
5	C/C++缺陷类型	空指针解引用	9	62
6	C/C++缺陷类型	并发	2	0
7	C/C++缺陷类型	除零	2	0
8	C/C++缺陷类型	数组问题	6	55
9	C/C++缺陷类型	不匹配的类型转换	11	33
10	C/C++缺陷类型	比较的问题	11	12
11	C/C++缺陷类型	整数溢出	2	0
12	C/C++缺陷类型	字符串问题	6	1
13	C/C++缺陷类型	注入问题	3	0
14	C/C++缺陷类型	返回值问题	10	134
15	C/C++缺陷类型	不可达代码	2	0
16	C/C++缺陷类型	代码质量-头文件问题	7	22
17	C/C++缺陷类型	代码质量-无用数据	8	0
18	C/C++缺陷类型	算术运算	12	53
19	C/C++缺陷类型	恒真恒假	2	0
20	C/C++缺陷类型	跨站攻击	1	0
21	C/C++缺陷类型	浮点数问题	2	0
22	C/C++缺陷类型	不安全密码	4	0
23	C/C++缺陷类型	日志管理	2	0
24	C/C++缺陷类型	资源泄露	3	0
25	C/C++缺陷类型	使用已释放的内存	4	0
26	C/C++缺陷类型	其他内存相关问题	10	21

4.3 人工复核结果统计表

缺陷级别	真实缺陷	误报缺陷	待确认缺陷	忽略缺陷	总计
致命	0	0	0	0	0
严重	0	0	456	0	456
一般	0	0	0	0	0
提示	0	0	0	0	0

强制	0	0	0	0	0
建议	0	0	0	0	0

4.4 代码质量度量统计表

变量描述	数量	简述
总行数	64695	源代码行数统计
总文件数	103	文件数量统计
总类个数	0	编译后中间文件个数
总方法个数	1636	所有的方法的个数
总有效代码行数	30435	有效代码行数>=总代码行数-注释行数-空行数 总行数 "c= a+b; //some comments" 如上面一行代码会同时归入有效代码行和注释行
总注释行数	19187	总注释行=文件头注释+方法头注释+方法内注释
文件头注释总行数	1358	文件头注释
函数声明注释总函数	5808	方法函数注释，域变量注释
方法内注释总行数	12021	方法内部的注释
总空行数	9371	空行数
注释比率	32.67	注释比率=注释行总数/代码总行数
声明注释比率	19.08	声明注释比率=头注释行数/有效代码
代码注释比例	39.49	代码注释比例=函数内注释行数/有效代码
总基本块数	0	基本块数
出度为 1 的基本块数	0	出度为 1 的基本块数
总操作数	540	加，减，乘，除，取余，与，或，异或等操作
参与操作符运算的变量数	429	参与操作符运算的变量数
程序出口个数	1	程序出口个数
扇入数比率	0.00	扇入数/函数数量
扇出数比率	0.00	扇入数/函数数量
扇入数	0	当前节点的所有被调用数量
扇出数	0	当前节点的所有调用数量
循环数	499	程序中所有的 loop 操作数量
圈复杂度	4.77	$(E-N+2)/\text{总方法数量}$

5 C/C++缺陷类型-初始化问题

5.1 分项汇总表

序号	C/C++缺陷类型-初始化问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4757	枚举元素定义中的初始化必须完整	9	严重
2	WK5723	结构体变量初始化的类型必须一致	54	严重

5.2 缺陷/安全漏洞详解

5.2.1 严重类

5.2.1.1 WK4757 枚举元素定义中的初始化必须完整

缺陷数量： 9 个		
缺陷描述： 枚举元素定义中的初始化必须完整。 注:规范的枚举元素初始化方法是,或初始化第一个元素,或初始化所有元素。		
修复意见： 无		
1	缺陷发生位置： /src/core/internal.h	行号： 148
	步骤描述： var_quote_e 枚举元素定义中的初始化必须完整	
	代码片段： 146: typedef var_t* (*var_eval_f)(var_t *args);	

	147: 148: enum var_quote_e { 149: QUOTE_DQ, // double quote, quotation mark 150: QUOTE_SQ, // single quote, apostrophe	
	污染路径: 无	
2	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 37
	步骤描述: stage_s 枚举元素定义中的初始化必须完整	
	代码片段: 35: 36: typedef enum stage_s stage_t; 37: enum stage_s { 38: STAGE_INITED, 39: STAGE_CONNECTED,	
	污染路径: 无	
3	缺陷发生位置: /src/core/internal.h	行号: 126
	步骤描述: var_e 枚举元素定义中的初始化必须完整	
	代码片段: 124: } 125: 126: enum var_e { 127: VAR_NULL, 128: VAR_BOOL,	
	污染路径: 无	
4	缺陷发生位置: /tests/c/test_threads.c	行号: 36
	步骤描述: thread_create_e 枚举元素定义中的初始化必须完整	
	代码片段: 34: 35: typedef enum thread_create_e thread_create_t; 36: enum thread_create_e { 37: USE_UNKNOWN, 38: USE_CreateThread,	

	污染路径: 无	
5	缺陷发生位置: /taosws/server/taoswsd.c	行号: 121
	步骤描述: twc_task_state_e 枚举元素定义中的初始化必须完整	
	代码片段: 119: typedef void (*task_routine)(twc_task_t *task); 120: 121: typedef enum twc_task_state_e { 122: TASK_STATE_NONE, 123: TASK_STATE_PENDING,	
	污染路径: 无	
6	缺陷发生位置: /benchmark/ob.c	行号: 168
	步骤描述: param_type_e 枚举元素定义中的初始化必须完整	
	代码片段: 166: 167: typedef enum param_type_e param_type_t; 168: enum param_type_e { 169: param_type_ms, 170: param_type_ms_str,	
	污染路径: 无	
7	缺陷发生位置: /src/core/internal.h	行号: 430
	步骤描述: backend_e 枚举元素定义中的初始化必须完整	
	代码片段: 428: 429: typedef enum backend_e backend_e; 430: enum backend_e { 431: BACKEND_TAOS, 432: BACKEND_TAOSWS,	
	污染路径: 无	
8	缺陷发生位置:	行号:

	/src/core/internal.h	814
	步骤描述: tables_type_e 枚举元素定义中的初始化必须完整	
	代码片段: 812: }; 813: 814: enum tables_type_e { 815: TABLES_FOR_GENERIC, 816: TABLES_FOR_CATALOGS,	
	污染路径: 无	
9	缺陷发生位置: /tests/taos/taos_regression.c	行号: 33
	步骤描述: stage_s 枚举元素定义中的初始化必须完整	
	代码片段: 31: 32: typedef enum stage_s stage_t; 33: enum stage_s { 34: STAGE_INITED, 35: STAGE_CONNECTED,	
	污染路径: 无	

5.2.1.2 WK5723 结构体变量初始化的类型必须一致

缺陷数量：54 个		
缺陷描述: 结构体变量的初值类型必须与结构体变量的定义类型一致。		
修复意见: 无		
1	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 1278
	步骤描述: 结构体变量的初始化要一致。	
	代码片段: 1276: ++i; 1277: if (i>=argc) break;	

	1278: arg.url = argv[i]; 1279: continue; 1280: }	
	污染路径: 无	
2	缺陷发生位置: /tests/c/test_threads.c	行号: 525
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 523: ++i; 524: if (i>=argc) break; 525: arg.dsn = argv[i]; 526: continue; 527: }	
	污染路径: 无	
3	缺陷发生位置: /tests/taos/taos_test.c	行号: 1224
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1222: if (e == TSDB_CODE_TSC_STMT_TBNAME_ERROR) { 1223: // insert into ? ... will result in TSDB_CODE_TSC_STMT_TBNAME_ERROR 1224: ctx->subtbl_required = 1; 1225: W("this is believed a subtbl insert statement"); 1226: r = 0;	
	污染路径: 无	
4	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 723
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 721: return -1; 722: } 723: cfg.conn = argv[i]; 724: continue; 725: }	

	污染路径: 无	
5	缺陷发生位置: /taosws/server/taoswsd.c	行号: 682
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 680: lws_write(wsi, (unsigned char*)buf_out, jout->nr_buf_out, (enum lws_write_protocol)flags); 681: jout->frame_chunks += 1; 682: jout->nr_buf_out = 0; 683: lws_callback_on_writable(wsi); 684: return 0;	
	污染路径: 无	
6	缺陷发生位置: /tests/c/test_threads.c	行号: 582
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 580: #endif /* { */ 581: 582: arg.name = argv[i]; 583: r = _run_with_arg(&arg, sqls, sizeof(sqls)/sizeof(sqls[0])); 584: if (r) return -1;	
	污染路径: 无	
7	缺陷发生位置: /common/ejson_parser_tab.c	行号: 113
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 111: if (!str) return; 112: if (str->str) str->str[0] = '\0'; 113: str->nr = 0; 114: } 115:	
	污染路径:	

	无	
8	缺陷发生位置: /tests/c/c_test.c	行号: 96
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 94: 95: CALL_SQLDisconnect(handles->hdbc); 96: handles->connected = 0; 97: } 98:	
	污染路径: 无	
9	缺陷发生位置: /taosws/server/taoswsd.c	行号: 430
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 428: step->str[0] = (uintptr_t)s; 429: step->str[1] = n; 430: step->cb = _twsql_serializer_step_on_str; 431: step->owner = serializer; 432:	
	污染路径: 无	
10	缺陷发生位置: /samples/c/demo_topic.c	行号: 352
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 350: ++i; 351: if (i>=argc) break; 352: arg.pwd = argv[i]; 353: continue; 354: }	
	污染路径: 无	
11	缺陷发生位置: /benchmark/ob.c	行号: 208
	步骤描述:	

	结构体变量的初始化要一致.	
	代码片段: 206: if (param_bind->buf) { 207: safe_free(param_bind->buf); 208: param_bind->sz = 0; 209: param_bind->nr = 0; 210: param_bind->ParameterValuePtr = NULL;	
	污染路径: 无	
12	缺陷发生位置: /tests/taos/taos_test.c	行号: 442
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 440: } 441: ctx->nr_cols = 0; 442: ctx->cols_described = 0; 443: } 444:	
	污染路径: 无	
13	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 709
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 707: odbc_conn_cfg_t cfg = {0}; 708: cfg.conn = "DSN=TAOS_ODBC_DSN;DATABASE=foo"; 709: cfg.rows = 32767; // INT16_MAX 710: 711: for (int i=1; i<argc; ++i) {	
	污染路径: 无	
14	缺陷发生位置: /tests/taos/taos_test.c	行号: 432
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 430: } 431: ctx->nr_tags = 0;	

	432: ctx->tags_described = 0; 433: } 434:	
	污染路径: 无	
15	缺陷发生位置: /tests/taos/taos_test.c	行号: 1070
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1068: D("field[#%d]: %s[%s, precision:%d, scale:%d, bytes:%d]", i+1, field->name, CALL_taos_data_type(field->type), field->precision, field->scale, field->bytes); 1069: } 1070: ctx->tags_described = 1; 1071: ctx->nr_tags = fieldNum; 1072: ctx->tags = fields;	
	污染路径: 无	
16	缺陷发生位置: /common/ejson_parser_tab.c	行号: 120
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 118: if (!bin) return; 119: if (bin->bin) bin->bin[0] = '\0'; 120: bin->nr = 0; 121: } 122:	
	污染路径: 无	
17	缺陷发生位置: /tests/c/c_test.c	行号: 1527
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1525: 1526: r = _test_topic_monitor(handles, &ds); 1527: ds.flag = 2; 1528:	

	1529: #ifdef _WIN32 /* { */	
	污染路径: 无	
18	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1288
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1286: 1287: task->on_release(task); 1288: task->state = TASK_STATE_NONE; 1289: 1290: session->task = NULL;	
	污染路径: 无	
19	缺陷发生位置: /tests/c/conformance_test.c	行号: 2913
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 2911: ++i; 2912: if (i>=argc) break; 2913: conn_arg.pwd = argv[i]; 2914: continue; 2915: }	
	污染路径: 无	
20	缺陷发生位置: /benchmark/ob.c	行号: 107
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 105: safe_free(col_binds->col_binds); 106: col_binds->nr = 0; 107: col_binds->sz = 0; 108: } 109:	
	污染路径: 无	

21	缺陷发生位置： /taosws/server/taoswsd.c	行号： 444
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 442: step->owner = serializer; 443: step->json = json; 444: step->cb = _twsql_json_serializer_step_on_json; 445: step->owner = serializer; 446:	
	污染路径： 无	
22	缺陷发生位置： /tests/taos/taosws_regression.c	行号： 1282
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 1280: } 1281: 1282: arg.name = argv[i]; 1283: r = start_by_arg(&arg); 1284: if (r) return -1;	
	污染路径： 无	
23	缺陷发生位置： /tests/c/conformance_test.c	行号： 2901
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 2899: ++i; 2900: if (i>=argc) break; 2901: conn_arg.dsn = argv[i]; 2902: continue; 2903: }	
	污染路径： 无	
24	缺陷发生位置： /tests/c/c_test.c	行号： 1524
	步骤描述： 结构体变量的初始化要一致.	

	代码片段: 1522: return -1; 1523: } 1524: ds.flag = 1; 1525: 1526: r = _test_topic_monitor(handles, &ds);	
	污染路径: 无	
25	缺陷发生位置: /samples/c/demo_topic.c	行号: 362
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 360: } 361: 362: arg.name = argv[i]; 363: r = _run_with_arg(&arg); 364: if (r) return -11;	
	污染路径: 无	
26	缺陷发生位置: /taosws/server/taoswsd.c	行号: 415
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 413: step->str[0] = (uintptr_t)s; 414: step->str[1] = n; 415: step->cb = _tws_json_serializer_step_on_raw_str; 416: step->owner = serializer; 417:	
	污染路径: 无	
27	缺陷发生位置: /tests/c/conformance_test.c	行号: 2895
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 2893: conn_arg_t conn_arg = {0}; 2894:	

	2895: conn_arg.odbc_type = TAOS_ODBC; 2896: 2897: for (int i=1; i<argc; ++i) {	
	污染路径: expanded from macro 'TAOS_ODBC'	
28	缺陷发生位置: /testes/c/test_threads.c	行号: 537
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 535: ++i; 536: if (i>=argc) break; 537: arg.pwd = argv[i]; 538: continue; 539: }	
	污染路径: 无	
29	缺陷发生位置: /benchmark/ob.c	行号: 209
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 207: safe_free(param_bind->buf); 208: param_bind->sz = 0; 209: param_bind->nr = 0; 210: param_bind->ParameterValuePtr = NULL; 211: }	
	污染路径: 无	
30	缺陷发生位置: /benchmark/ob.c	行号: 457
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 455: safe_free(param_binds->param_binds); 456: } 457: param_binds->sz = 0; 458: param_binds->nr = 0; 459: if (param_binds->ParamStatusPtr) {	

	污染路径: 无	
31	缺陷发生位置: /benchmark/ob.c	行号: 106
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 104: } 105: safe_free(col_binds->col_binds); 106: col_binds->nr = 0; 107: col_binds->sz = 0; 108: }	
	污染路径: 无	
32	缺陷发生位置: /samples/c/demo_topic.c	行号: 358
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 356: ++i; 357: if (i>=argc) break; 358: arg.connstr = argv[i]; 359: continue; 360: }	
	污染路径: 无	
33	缺陷发生位置: /src/odbc.c	行号: 158
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 156: static void _init_once(void) 157: { 158: _global.taos_odbc_debug_flex = check_env_bool("TAOS_ODBC_DEBUG_FLEX"); 159: _global.taos_odbc_debug_bison = check_env_bool("TAOS_ODBC_DEBUG_BISON"); 160: _init_charsets();	
	污染路径: 无	

34	缺陷发生位置： /tests/c/conformance_test.c	行号： 2907
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 2905: ++i; 2906: if (i>=argc) break; 2907: conn_arg.uid = argv[i]; 2908: continue; 2909: }	
	污染路径： 无	
35	缺陷发生位置： /tests/c/odbc_test.c	行号： 1444
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 1442: ++i; 1443: if (i>=argc) break; 1444: conn_arg.uid = argv[i]; 1445: continue; 1446: }	
	污染路径： 无	
36	缺陷发生位置： /samples/c/demo_topic.c	行号： 340
	步骤描述： 结构体变量的初始化要一致.	
	代码片段： 338: ++i; 339: if (i>=argc) break; 340: arg.dsn = argv[i]; 341: continue; 342: }	
	污染路径： 无	
37	缺陷发生位置： /src/utlis/utlis.c	行号： 331
	步骤描述： 结构体变量的初始化要一致.	

	代码片段: 329: node->match = NULL; 330: node->base = NULL; 331: node->nr = 0; 332: } 333:	
	污染路径: 无	
38	缺陷发生位置: /tests/c/odbc_test.c	行号: 1460
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1458: } 1459: if (strcmp(argv[i], "--non-taos") == 0) { 1460: conn_arg.non_taos = 1; 1461: continue; 1462: }	
	污染路径: 无	
39	缺陷发生位置: /common/ejson_parser_tab.c	行号: 77
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 75: { 76: if (!internal) return; 77: internal->nr = 0; 78: if (internal->buf) internal->buf[0] = '\0'; 79: }	
	污染路径: 无	
40	缺陷发生位置: /tests/c/odbc_test.c	行号: 1450
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1448: ++i; 1449: if (i>=argc) break; 1450: conn_arg.pwd = argv[i];	

	1451: continue; 1452: }	
	污染路径: 无	
41	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 675
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 673: int r = 0; 674: 675: PROFILE(r = _do_run(hstmt, cfg)); 676: 677: return r ? -1 : 0;	
	污染路径: expanded from macro 'PROFILE'	
42	缺陷发生位置: /common/logger.c	行号: 292
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 290: tick = (double)ticks.QuadPart/freq.QuadPart; 291: #else /* }{ */ 292: struct timespec ts = {0}; 293: 294: clock_gettime(CLOCK_MONOTONIC_RAW, &ts);	
	污染路径: 无	
43	缺陷发生位置: /common/ejson_parser_tab.c	行号: 89
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 87: internal->buf = NULL; 88: } 89: internal->cap = 0; 90: } 91:	
	污染路径:	

	无	
44	缺陷发生位置: /tests/c/conformance_test.c	行号: 2919
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 2917: ++i; 2918: if (i>=argc) break; 2919: conn_arg.connstr = argv[i]; 2920: continue; 2921: }	
	污染路径: 无	
45	缺陷发生位置: /common/ejson_parser_tab.c	行号: 131
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 129: str->str = NULL; 130: } 131: str->cap = 0; 132: } 133:	
	污染路径: 无	
46	缺陷发生位置: /tests/c/c_test.c	行号: 161
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 159: if (r) break; 160: 161: handles->connected = 1; 162: } 163:	
	污染路径: 无	
47	缺陷发生位置: /tests/c/odbc_test.c	行号: 1456
	步骤描述:	

	结构体变量的初始化要一致.	
	代码片段: <pre> 1454: ++i; 1455: if (i>=argc) break; 1456: conn_arg.connstr = argv[i]; 1457: continue; 1458: }</pre>	
	污染路径: 无	
48	缺陷发生位置:	行号:
	/samples/c/demo_topic.c	346
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 344: ++i; 345: if (i>=argc) break; 346: arg.uid = argv[i]; 347: continue; 348: }</pre>	
49	污染路径: 无	
	缺陷发生位置:	行号:
	/tests/c/test_threads.c	502
	步骤描述: 结构体变量的初始化要一致.	
50	代码片段: <pre> 500: arg.thread_create = USE_CreateThread; 501: #else /* } { */ 502: arg.thread_create = USE_pthread_create; 503: #endif /* } */ 504:</pre>	
	污染路径: 无	
	缺陷发生位置:	行号:
	/tests/taos/taos_test.c	1088
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: <pre> 1086: D("co[##d]: %s[%s, precision:%d, scale:%d, bytes:%d]", i+1, col->name, CALL_taos_data_type(col->type),</pre>	

	col->precision, col->scale, col->bytes); 1087: } 1088: ctx->cols_described = 1; 1089: ctx->nr_cols = colNum; 1090: ctx->cols = cols;	
	污染路径: 无	
51	缺陷发生位置: /tests/c/test_threads.c	行号: 531
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 529: ++i; 530: if (i>=argc) break; 531: arg.uid = argv[i]; 532: continue; 533: }	
	污染路径: 无	
52	缺陷发生位置: /benchmark/taos_benchmark.c	行号: 602
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 600: if (r) return -1; 601: 602: PROFILE(r = do_benchmark_case0(stmt, cfg)); 603: 604: return r ? -1 : 0;	
	污染路径: expanded from macro 'PROFILE'	
53	缺陷发生位置: /src/odbc.c	行号: 159
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 157: { 158: _global.taos_odbc_debug_flex = check_env_bool("TAOS_ODBC_DEBUG_FLEX"); 159: _global.taos_odbc_debug_bison =	

	check_env_bool("TAOS_ODBC_DEBUG_BISON"); 160: _init_charsets(); 161: }	
	污染路径: 无	
54	缺陷发生位置: /tests/c/odbc_test.c	行号: 1438
	步骤描述: 结构体变量的初始化要一致.	
	代码片段: 1436: ++i; 1437: if (i>=argc) break; 1438: conn_arg.dsn = argv[i]; 1439: continue; 1440: }	
	污染路径: 无	

6 C/C++缺陷类型-空指针解引用

6.1 分项汇总表

序号	C/C++缺陷类型-空指针解引用	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4691	动态分配的指针第一次使用前必须进行是否为 NULL 的判别	6	严重
2	WK4692	空指针必须使用 NULL,禁止使用整数 0	28	严重

3	WK4871	宏 NULL 应是唯一允许的整数空指针常量形式	28	严重
---	--------	-------------------------	----	----

6.2 缺陷/安全漏洞详解

6.2.1 严重类

6.2.1.1 WK4691 动态分配的指针第一次使用前必须进行是否为

NULL 的判别

缺陷数量：6 个		
缺陷描述： 动态分配的指针第一次使用前必须进行是否为 NULL 的判别。		
修复意见： 无		
1	缺陷发生位置： /tests/taos/taos_test.c	行号： 497
	步骤描述： 动态分配的指针 is_null 第一次使用前必须进行是否为 NULL 的判别	
	代码片段： 495: int32_t *buffer = (int32_t*)malloc(rows * sizeof(*buffer)); 496: int32_t *length = (int32_t*)malloc(rows * sizeof(*length)); 497: char *is_null = (char*)malloc(rows * sizeof(*is_null)); 498: 499: if (!buffer buffers_append(&ctx->buffers, (char*)buffer)) {	
	污染路径： 无	
2	缺陷发生位置： /tests/c/odbc_test.c	行号： 878

	步骤描述: 动态分配的指针 p1 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 876: { 877: int r = 0; 878: char *p1 = (char*)malloc(bytes_per_row * rows); 879: SQLEN *p2 = (SQLEN*)malloc(sizeof(*p2) * rows); 880: if (p1 && p2) {	
	污染路径: 无	
3	缺陷发生位置: /tests/taos/taos_test.c	行号: 460
	步骤描述: 动态分配的指针 is_null 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 458: int64_t *buffer = (int64_t*)malloc(rows * sizeof(*buffer)); 459: int32_t *length = (int32_t*)malloc(rows * sizeof(*length)); 460: char *is_null = (char*)malloc(rows * sizeof(*is_null)); 461: 462: if (!buffer buffers_append(&ctx->buffers, (char*)buffer)) {	
	污染路径: 无	
4	缺陷发生位置: /tests/taos/taos_test.c	行号: 534
	步骤描述: 动态分配的指针 is_null 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 532: int32_t *buffer = (int32_t*)malloc(rows * col->bytes); 533: int32_t *length = (int32_t*)malloc(rows * sizeof(*length)); 534: char *is_null = (char*)malloc(rows * sizeof(*is_null)); 535: 536: if (!buffer buffers_append(&ctx->buffers, (char*)buffer)) {	

	污染路径: 无	
5	缺陷发生位置: /tests/taos/taos_test.c	行号: 569
	步骤描述: 动态分配的指针 buffer 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 567: static int _prepare_mb_as_tsdb_double(executes_ctx_t *ctx, TAOS_MULTI_BIND *col_mb, int rows) 568: { 569: double *buffer = (double*)malloc(rows * sizeof(*buffer)); 570: int32_t *length = (int32_t*)malloc(rows * sizeof(*length)); 571: char *is_null = (char*)malloc(rows * sizeof(*is_null));	
	污染路径: 无	
6	缺陷发生位置: /tests/taos/taos_test.c	行号: 571
	步骤描述: 动态分配的指针 is_null 第一次使用前必须进行是否为 NULL 的判别	
	代码片段: 569: double *buffer = (double*)malloc(rows * sizeof(*buffer)); 570: int32_t *length = (int32_t*)malloc(rows * sizeof(*length)); 571: char *is_null = (char*)malloc(rows * sizeof(*is_null)); 572: 573: if (!buffer buffers_append(&ctx->buffers, (char*)buffer)) {	
	污染路径: 无	

6.2.1.2 WK4692 空指针必须使用 NULL,禁止使用整数 0

缺陷数量: 28 个
缺陷描述:

空指针必须使用 NULL,禁止使用整数 0。		
修复意见： 无		
1	缺陷发生位置： /benchmark/odbc_benchmark.c	行号： 633
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 631: SFREE(data[i]); 632: } 633: SFREE(data); 634: } 635:	
	污染路径： expanded from macro 'SFREE' expanded from macro 'NULL'	
2	缺陷发生位置： /taosws/server/taoswsd.c	行号： 584
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 582: tw_err_t *err = lws_container_of(d, tw_err_t, node); 583: tw_err_release(err); 584: SAFE_FREE(err); 585: 586: return 0;	
	污染路径： expanded from macro 'SAFE_FREE' expanded from macro 'NULL'	
3	缺陷发生位置： /tests/c/api_test.c	行号： 1268
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 1266: return -1; 1267: } 1268: char *end = 0; 1269: errno = 0; 1270: long v = strtol(argv[*i], &end, 0);	
	污染路径：	

	无	
4	缺陷发生位置： /samples/c/demo_topic.c	行号： 330
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 328: int r = 0; 329: 330: arg_t arg = {0}; 331: 332: for (int i=1; i<argc; ++i) {	
	污染路径： 无	
5	缺陷发生位置： /benchmark/odbc_benchmark.c	行号： 707
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 705: int r = 0; 706: 707: odbc_conn_cfg_t cfg = {0}; 708: cfg.conn = "DSN=TAOS_ODBC_DSN;DATABASE=foo"; 709: cfg.rows = 32767; // INT16_MAX	
	污染路径： 无	
6	缺陷发生位置： /common/ejson_parser_tab.c	行号： 890
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 888: if (pp) { 889: if (curr > prev) { 890: _SNPRINTF("%.*s", (int)(curr - prev), prev); 891: } 892: _SNPRINTF("\\%c", escape2_replace[pp-escape2]);	
	污染路径： expanded from macro '_SNPRINTF' expanded from macro '_ADJUST' expanded from macro 'NULL'	
7	缺陷发生位置：	行号：

	/benchmark/ob.c	455
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 453: } 454: if (param_binds->param_binds) { 455: safe_free(param_binds->param_binds); 456: } 457: param_binds->sz = 0;	
	污染路径: expanded from macro 'safe_free' expanded from macro 'NULL'	
8	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1390
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1388: } 1389: 1390: SAFE_FREE(workers->workers); 1391: } 1392:	
	污染路径: expanded from macro 'SAFE_FREE' expanded from macro 'NULL'	
9	缺陷发生位置: /common/ejson_parser_tab.c	行号: 892
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 890: _SPRINTF("%.s", (int)(curr - prev), prev); 891: } 892: _SPRINTF("\\%c", escape2_replace[pp-escape2]); 893: prev = ++curr; 894: continue;	
	污染路径: expanded from macro '_SPRINTF' expanded from macro '_ADJUST' expanded from macro 'NULL'	
10	缺陷发生位置:	行号:

	/tests/taos/taosws_regression.c	1267
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1265: int r = 0; 1266: 1267: arg_t arg = {0}; 1268: arg.url = "http://" WS_FOR_TEST; 1269:	
	污染路径: 无	
11	缺陷发生位置: /benchmark/ob.c	行号: 697
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 695: static int run_query(handles_t *handles, int i, int argc, char *argv[]) 696: { 697: col_binds_t col_binds = {0}; 698: int r = run_query_with_col_binds(handles, &col_binds, i, argc, argv); 699: col_binds_release(&col_binds);	
	污染路径: 无	
12	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1068
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1066: stmt->stmt = NULL; 1067: } 1068: SAFE_FREE(stmt->sql); 1069: } 1070:	
	污染路径: expanded from macro 'SAFE_FREE' expanded from macro 'NULL'	
13	缺陷发生位置: /tests/taos/taos_regression.c	行号: 928

	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 926: if (r) break; 927: if (numOfRows == 0) break; 928: raw_block_t raw_block = {0}; 929: if (raw_block_init(&raw_block, numOfRows, pData)) return -1; 930: if (fout) {	
	污染路径: 无	
14	缺陷发生位置: /tests/c/sqlite3_test.c	行号: 551
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 549: (void)argc; 550: (void)argv; 551: srand(time(0)); 552: 553: int r = test();	
	污染路径: 无	
15	缺陷发生位置: /taosws/server/taoswsd.c	行号: 550
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 548: tws_io_t *io = lws_container_of(d, tws_io_t, node); 549: tws_session_release_io(session, io); 550: SAFE_FREE(io); 551: 552: return 0;	
	污染路径: expanded from macro 'SAFE_FREE' expanded from macro 'NULL'	
16	缺陷发生位置: /src/odbc.c	行号: 107
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	

	代码片段: <pre> 105: snprintf(_global.image_name, sizeof(_global.image_name), "%s", p); 106: #else /* }{ */ 107: Dl_info info = {0}; 108: dladdr(_init_image_path_once, &info); 109: snprintf(_global.image_path, sizeof(_global.image_path), "%s", info.dli_fname); </pre>	
	污染路径: 无	
17	缺陷发生位置: /benchmark/ob.c	行号: 207
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: <pre> 205: { 206: if (param_bind->buf) { 207: safe_free(param_bind->buf); 208: param_bind->sz = 0; 209: param_bind->nr = 0; </pre>	
	污染路径: expanded from macro 'safe_free' expanded from macro 'NULL'	
18	缺陷发生位置: /tests/c/edge_cases_test.c	行号: 563
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: <pre> 561: static int test(void) 562: { 563: srand((unsigned int)time(0)); 564: 565: CHECK(!test_connect(NULL, "xTAOS_ODBC_DSN", NULL, NULL)); </pre>	
	污染路径: 无	
19	缺陷发生位置: /tests/c/test_threads.c	行号: 371
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	

	代码片段: 369: r = _thread_routine((const arg_t*)arg); 370: if (r) return (void*)-1; 371: return 0; 372: } 373:	
	污染路径: 无	
20	缺陷发生位置: /tests/c/api_test.c	行号: 1289
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1287: return -1; 1288: } 1289: char *end = 0; 1290: errno = 0; 1291: long v = strtol(argv[*i], &end, 0);	
	污染路径: 无	
21	缺陷发生位置: /tests/c/odbc_test.c	行号: 1432
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1430: (void)argv; 1431: 1432: conn_arg_t conn_arg = {0}; 1433: 1434: for (int i=1; i<argc; ++i) {	
	污染路径: 无	
22	缺陷发生位置: /common/ejson_parser_tab.c	行号: 881
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 879: if (strchr(escape1, c)) { 880: if (curr > prev) { 881: _SNPRINTF("%.*s", (int)(curr - prev), prev);	

	882: } 883: _SNPRINTF("\\%c", c);	
	污染路径: expanded from macro '_SNPRINTF' expanded from macro '_ADJUST' expanded from macro 'NULL'	
23	缺陷发生位置: /common/ejson_parser_tab.c	行号: 869
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 867: int n = 0; 868: 869: _SNPRINTF("%c", ''); 870: 871: if (str && nr > 0) {	
	污染路径: expanded from macro '_SNPRINTF' expanded from macro '_ADJUST' expanded from macro 'NULL'	
24	缺陷发生位置: /common/ejson_parser_tab.c	行号: 883
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 881: _SNPRINTF("%.s", (int)(curr - prev), prev); 882: } 883: _SNPRINTF("\\%c", c); 884: prev = ++curr; 885: continue;	
	污染路径: expanded from macro '_SNPRINTF' expanded from macro '_ADJUST' expanded from macro 'NULL'	
25	缺陷发生位置: /tests/c/conformance_test.c	行号: 2893
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2891: static int test_conn(int argc, char *argv[], SQLHANDLE	

	hconn) 2892: { 2893: conn_arg_t conn_arg = {0}; 2894: 2895: conn_arg.odbc_type = TAOS_ODBC;	
	污染路径: 无	
26	缺陷发生位置: /testsc/test_threads.c	行号: 493
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 491: { 492: int r = 0; 493: arg_t arg = {0}; 494: arg.dsn = dsn; 495: arg.main = 1;	
	污染路径: 无	
27	缺陷发生位置: /benchmark/ob.c	行号: 105
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 103: col_bind_release(col_bind); 104: } 105: safe_free(col_binds->col_binds); 106: col_binds->nr = 0; 107: col_binds->sz = 0;	
	污染路径: expanded from macro 'safe_free' expanded from macro 'NULL'	
28	缺陷发生位置: /taosws/server/taoswsd.c	行号: 567
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 565: lws_dll2_remove(d); 566: tws_json_serializer_step_t *step = lws_container_of(d, tws_json_serializer_step_t, node);	

	567: SAFE_FREE(step); 568: 569: return 0;
	污染路径: expanded from macro 'SAFE_FREE' expanded from macro 'NULL'

6.2.1.3 WK4871 宏 NULL 应是唯一允许的整数空指针常量形式

缺陷数量：28 个		
缺陷描述： 如果出现在以下任何上下文中，则值为 0 的整数常量表达式应从宏 NULL 的展开中导出： • 作为分配给指针的值； • 作为 == 或 != 的操作数另一个操作数是指针的运算符； • 作为 ? 的第二个操作数：第三个操作数是指针的运算符； • 作为 ? 的第三个操作数：第二个操作数是指针的运算符。 忽略空格和任何周围的括号，任何此类整数常量表达式都应表示 NULL 的整个扩展。 注意：允许使用 (void*) 0 形式的空指针常量，无论它是否从 null 扩展而来。 理论基础 使用 NULL 而不是 0 可以清楚地表明需要一个空指针常量。		
修复意见： 无		
1	缺陷发生位置： /common/ejson_parser_tab.c	行号： 890
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段： 888: if (pp) { 889: if (curr > prev) { 890: _SNPRINTF("%.*s", (int)(curr - prev), prev); 891: } 892: _SNPRINTF("\\%c", escape2_replace[pp-escape2]);	
	污染路径： expanded from macro '_SNPRINTF' expanded from macro '_ADJUST' expanded from macro 'NULL'	
2	缺陷发生位置： /common/ejson_parser_tab.c	行号： 883
	步骤描述： 空指针必须使用 NULL,禁止使用整数 0	
	代码片段：	

	<pre> 881: _SNPRINTF("%.s", (int)(curr - prev), prev); 882: } 883: _SNPRINTF("\\%c", c); 884: prev = ++curr; 885: continue; </pre>	
	污染路径: expanded from macro '_SNPRINTF' expanded from macro '_ADJUST' expanded from macro 'NULL'	
3	缺陷发生位置: /benchmark/ob.c	行号: 207
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: <pre> 205: { 206: if (param_bind->buf) { 207: safe_free(param_bind->buf); 208: param_bind->sz = 0; 209: param_bind->nr = 0; </pre>	
	污染路径: expanded from macro 'safe_free' expanded from macro 'NULL'	
4	缺陷发生位置: /taosws/server/taoswsd.c	行号: 584
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: <pre> 582: tws_err_t *err = lws_container_of(d, tws_err_t, node); 583: tws_err_release(err); 584: SAFE_FREE(err); 585: 586: return 0; </pre>	
	污染路径: expanded from macro 'SAFE_FREE' expanded from macro 'NULL'	
5	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 1267
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段:	

	1265: int r = 0; 1266: 1267: arg_t arg = {0}; 1268: arg.url = "http://" WS_FOR_TEST; 1269:	
	污染路径: 无	
6	缺陷发生位置: /taosws/server/taoswsd.c	行号: 567
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 565: lws_dll2_remove(d); 566: tws_json_serializer_step_t *step = lws_container_of(d, tws_json_serializer_step_t, node); 567: SAFE_FREE(step); 568: 569: return 0;	
	污染路径: expanded from macro 'SAFE_FREE' expanded from macro 'NULL'	
7	缺陷发生位置: /benchmark/ob.c	行号: 697
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 695: static int run_query(handles_t *handles, int i, int argc, char *argv[]) 696: { 697: col_binds_t col_binds = {0}; 698: int r = run_query_with_col_binds(handles, &col_binds, i, argc, argv); 699: col_binds_release(&col_binds);	
	污染路径: 无	
8	缺陷发生位置: /benchmark/ob.c	行号: 455
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段:	

	453: } 454: if (param_binds->param_binds) { 455: safe_free(param_binds->param_binds); 456: } 457: param_binds->sz = 0;	
	污染路径: expanded from macro 'safe_free' expanded from macro 'NULL'	
9	缺陷发生位置: /common/ejson_parser_tab.c	行号: 881
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 879: if (strchr(escape1, c)) { 880: if (curr > prev) { 881: _SPRINTF("%.*s", (int)(curr - prev), prev); 882: } 883: _SPRINTF("\\%c", c);	
	污染路径: expanded from macro '_SPRINTF' expanded from macro '_ADJUST' expanded from macro 'NULL'	
10	缺陷发生位置: /tests/c/sqlite3_test.c	行号: 551
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 549: (void)argc; 550: (void)argv; 551: srand(time(0)); 552: 553: int r = test();	
	污染路径: 无	
11	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 633
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 631: SFREE(data[i]);	

	632: } 633: SFREE(data); 634: } 635:	
	污染路径: expanded from macro 'SFREE' expanded from macro 'NULL'	
12	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1068
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1066: stmt->stmt = NULL; 1067: } 1068: SAFE_FREE(stmt->sql); 1069: } 1070:	
	污染路径: expanded from macro 'SAFE_FREE' expanded from macro 'NULL'	
13	缺陷发生位置: /tests/c/api_test.c	行号: 1268
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1266: return -1; 1267: } 1268: char *end = 0; 1269: errno = 0; 1270: long v = strtol(argv[*i], &end, 0);	
	污染路径: 无	
14	缺陷发生位置: /common/ejson_parser_tab.c	行号: 892
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 890: _SPRINTF("%.*s", (int)(curr - prev), prev); 891: } 892: _SPRINTF("\\%c", escape2_replace[pp-escape2]);	

	893: prev = ++curr; 894: continue;	
	污染路径: expanded from macro '_SNPRINTF' expanded from macro '_ADJUST' expanded from macro 'NULL'	
15	缺陷发生位置: /tests/c/api_test.c	行号: 1289
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1287: return -1; 1288: } 1289: char *end = 0; 1290: errno = 0; 1291: long v = strtol(argv[*i], &end, 0);	
	污染路径: 无	
16	缺陷发生位置: /tests/c/edge_cases_test.c	行号: 563
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 561: static int test(void) 562: { 563: srand((unsigned int)time(0)); 564: 565: CHECK(!test_connect(NULL, "xTAOS_ODBC_DSN", NULL, NULL));	
	污染路径: 无	
17	缺陷发生位置: /benchmark/ob.c	行号: 105
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 103: col_bind_release(col_bind); 104: } 105: safe_free(col_binds->col_binds); 106: col_binds->nr = 0;	

	107: col_binds->sz = 0;	
	污染路径: expanded from macro 'safe_free' expanded from macro 'NULL'	
18	缺陷发生位置: /tests/c/conformance_test.c	行号: 2893
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 2891: static int test_conn(int argc, char *argv[], SQLHANDLE hconn) 2892: { 2893: conn_arg_t conn_arg = {0}; 2894: 2895: conn_arg.odbc_type = TAOS_ODBC;	
	污染路径: 无	
19	缺陷发生位置: /tests/c/test_threads.c	行号: 493
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 491: { 492: int r = 0; 493: arg_t arg = {0}; 494: arg.dsn = dsn; 495: arg.main = 1;	
	污染路径: 无	
20	缺陷发生位置: /samples/c/demo_topic.c	行号: 330
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 328: int r = 0; 329: 330: arg_t arg = {0}; 331: 332: for (int i=1; i<argc; ++i) {	

	污染路径: 无	
21	缺陷发生位置: /common/ejson_parser_tab.c	行号: 869
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 867: int n = 0; 868: 869: _SNPRINTF("%c", ""); 870: 871: if (str && nr > 0) {	
	污染路径: expanded from macro '_SNPRINTF' expanded from macro '_ADJUST' expanded from macro 'NULL'	
22	缺陷发生位置: /src/odbc.c	行号: 107
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 105: snprintf(_global.image_name, sizeof(_global.image_name), "%s", p); 106: #else /* }{ */ 107: DI_info info = {0}; 108: dladdr(_init_image_path_once, &info); 109: snprintf(_global.image_path, sizeof(_global.image_path), "%s", info.dli_fname);	
	污染路径: 无	
23	缺陷发生位置: /tests/taos/taos_regression.c	行号: 928
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 926: if (r) break; 927: if (numOfRows == 0) break; 928: raw_block_t raw_block = {0}; 929: if (raw_block_init(&raw_block, numOfRows, pData)) return -1; 930: if (fout) {	

	污染路径: 无	
24	缺陷发生位置: /tests/c/odbc_test.c	行号: 1432
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1430: (void)argv; 1431: 1432: conn_arg_t conn_arg = {0}; 1433: 1434: for (int i=1; i<argc; ++i) {	
	污染路径: 无	
25	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 707
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 705: int r = 0; 706: 707: odbc_conn_cfg_t cfg = {0}; 708: cfg.conn = "DSN=TAOS_ODBC_DSN;DATABASE=foo"; 709: cfg.rows = 32767; // INT16_MAX	
	污染路径: 无	
26	缺陷发生位置: /taosws/server/taoswsd.c	行号: 550
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 548: tws_io_t *io = lws_container_of(d, tws_io_t, node); 549: tws_session_release_io(session, io); 550: SAFE_FREE(io); 551: 552: return 0;	
	污染路径: expanded from macro 'SAFE_FREE' expanded from macro 'NULL'	

27	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1390
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 1388: } 1389: 1390: SAFE_FREE(workers->workers); 1391: } 1392:	
	污染路径: expanded from macro 'SAFE_FREE' expanded from macro 'NULL'	
28	缺陷发生位置: /tests/c/test_threads.c	行号: 371
	步骤描述: 空指针必须使用 NULL,禁止使用整数 0	
	代码片段: 369: r = _thread_routine((const arg_t*)arg); 370: if (r) return (void*)-1; 371: return 0; 372: } 373:	
	污染路径: 无	

7 C/C++缺陷类型-数组问题

7.1 分项汇总表

序号	C/C++缺陷类型-数组问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4672	数组定义禁止	55	严重

		没有显式的边界限定		
--	--	-----------	--	--

7.2 缺陷/安全漏洞详解

7.2.1 严重类

7.2.1.1 WK4672 数组定义禁止没有显式的边界限定

缺陷数量： 55 个		
缺陷描述： 数组定义禁止没有显式的边界限定。		
修复意见： 无		
1	缺陷发生位置： /tests/c/conformance_test.c	行号： 888
	步骤描述： 数组 view_sqls 定义的没有显式的边界限定	
	代码片段： 886: // test view 887: if (_test_with_enterprise) { 888: struct { 889: const char *sql; 890: } view_sqls[] = {	
	污染路径： 无	
2	缺陷发生位置： /tests/taos/taos_regression.c	行号： 853
	步骤描述： 数组 sqls 定义的没有显式的边界限定	
	代码片段： 851: } 852: 853: const sql_t sqls[] = { 854: {"drop database if exists foo", __LINE__}, 855: {"create database if not exists foo", __LINE__},	

	污染路径: 无	
3	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 308
	步骤描述: 数组_cases 定义的没有显式的边界限定	
	代码片段: 306: if (stage != STAGE_CONNECTED) return 0; 307: 308: const struct { 309: const char *sql; 310: int __line__;	
	污染路径: 无	
4	缺陷发生位置: /tests/c/odbc_test.c	行号: 2443
	步骤描述: 数组 cases 定义的没有显式的边界限定	
	代码片段: 2441: static int test_cases_get_data(SQLHANDLE henv) 2442: { 2443: struct { 2444: const char *conn_str; 2445: int odbc;	
	污染路径: 无	
5	缺陷发生位置: /src/tests/basic.c	行号: 825
	步骤描述: 数组_cases 定义的没有显式的边界限定	
	代码片段: 823: { 824: #define RECORD(x,y) {__LINE__, x, y} 825: struct { 826: int line; 827: const char *url;	
	污染路径: 无	
6	缺陷发生位置: /src/core/topic.c	行号: 240

	步骤描述: 数组 pseudos 定义的没有显式的边界限定	
	代码片段: 238: #endif /*] */ 239: 240: const char *pseudos[] = { 241: "_topic_name", 242: "_db_name",	
	污染路径: 无	
7	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 225
	步骤描述: 数组 _cases 定义的没有显式的边界限定	
	代码片段: 223: if (stage != STAGE_CONNECTED) return 0; 224: 225: const struct { 226: const char *sql; 227: int __line__;	
	污染路径: 无	
8	缺陷发生位置: /src/core/columns.c	行号: 478
	步骤描述: 数组 supported 定义的没有显式的边界限定	
	代码片段: 476: } 477: 478: static const struct { 479: const char *s; 480: int tsdb_type;	
	污染路径: 无	
9	缺陷发生位置: /tests/taos/taos_regression.c	行号: 870
	步骤描述: 数组 s_exps 定义的没有显式的边界限定	
	代码片段: 868:	

	869: #define RECORD(x) {x, __LINE__} 870: const exp_t s_exps[] = { 871: RECORD("1688693481425"), 872: RECORD("1688693481429"),	
	污染路径: 无	
10	缺陷发生位置: /tests/c/conformance_test.c	行号: 983
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 981: const char *ColumnName; 982: 983: struct { 984: const char *sql; 985: } sqls[] = {	
	污染路径: 无	
11	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 419
	步骤描述: 数组_cases 定义的没有显式的边界限定	
	代码片段: 417: if (stage != STAGE_STATEMENT) return 0; 418: 419: const struct { 420: const char *sql; 421: int __line__;	
	污染路径: 无	
12	缺陷发生位置: /src/core/conn.c	行号: 492
	步骤描述: 数组 convs 定义的没有显式的边界限定	
	代码片段: 490: } 491: if (0) { 492: struct { 493: const char *from; 494: const char *to;	

	污染路径: 无	
13	缺陷发生位置: /common/logger.c	行号: 198
	步骤描述: 数组 cfgs 定义的没有显式的边界限定	
	代码片段: 196: static void _init_system_logger(void) 197: { 198: struct { 199: const char *name; 200: void (*init)(void);	
	污染路径: 无	
14	缺陷发生位置: /tests/c/odbc_test.c	行号: 2560
	步骤描述: 数组 cases 定义的没有显式的边界限定	
	代码片段: 2558: static int test_cases_prepare(SQLHANDLE henv) 2559: { 2560: struct { 2561: const char *conn_str; 2562: int odbc;	
	污染路径: 无	
15	缺陷发生位置: /src/tests/basic.c	行号: 415
	步骤描述: 数组 _cases 定义的没有显式的边界限定	
	代码片段: 413: { 414: #define RECORD(x, y) {x, y, __LINE__} 415: const struct { 416: const char *text; 417: const char *match;	
	污染路径: 无	
16	缺陷发生位置:	行号:

	/tests/c/odbc_test.c		2388
	步骤描述: 数组 sqls 定义的没有显式的边界限定		
	代码片段: 2386: #define LOCAL_NAME "我 shi4 中国人" 2387: 2388: struct { 2389: const char *sql; 2390: int odbc;		
	污染路径: 无		
17	缺陷发生位置: /tests/c/api_test.c		行号: 1212
	步骤描述: 数组_target_types 定义的没有显式的边界限定		
	代码片段: 1210: // { 1211: #define R(x, y) {#x, x, y} 1212: static struct { 1213: const char *name; 1214: SQLSMALLINT TargetType;		
	污染路径: 无		
18	缺陷发生位置: /tests/c/test_threads.c		行号: 77
	步骤描述: 数组 sqls 定义的没有显式的边界限定		
	代码片段: 75: 76: if (1) { 77: const char *sqls[] = { 78: "select * from information_schema.ins_configs", 79: };		
	污染路径: 无		
19	缺陷发生位置: /tests/taos/taos_test.c		行号: 2475
	步骤描述: 数组 sqls 定义的没有显式的边界限定		
	代码片段:		

	2473: { 2474: int r = 0; 2475: const char *sqls[] = { 2476: "show databases", 2477: "drop database if exists foo",	
	污染路径: 无	
20	缺陷发生位置: /tests/c/c_test.c	行号: 1033
	步骤描述: 数组 name 定义的没有显式的边界限定	
	代码片段: 1031: 1032: sql = "select name from foo.test_types"; 1033: const char name[] = "test123"; 1034: r = CHECK_WITH_VALUES(handles, 1, sql, 1, 1, name); 1035: if (r) return -1;	
	污染路径: 无	
21	缺陷发生位置: /common/logger.c	行号: 122
	步骤描述: 数组_levels 定义的没有显式的边界限定	
	代码片段: 120: } 121: #define RECORD(x) {#x, LOGGER_##x} 122: struct { 123: const char *name; 124: int level;	
	污染路径: 无	
22	缺陷发生位置: /tests/cpp/SQLBindCol_ref.cpp	行号: 309
	步骤描述: 数组 exp 定义的没有显式的边界限定	
	代码片段: 307: A(SUCCEEDED(rc), ""); 308: 309: const char *exp[][5] = { 310: {"[2022-09-11 09:57:28.752]", "[name1]",	

	"[20]", "[male]", "[中国人]"}, 311: {"[2022-09-11 09:57:29.753]", "[name2]", "[30]", "[female]", "[苏州人]"},	
	污染路径: 无	
23	缺陷发生位置: /src/tests/basic.c	行号: 73
	步骤描述: 数组_cases 定义的没有显式的边界限定	
	代码片段: 71: // param.ctx.debug_bison = 1; 72: 73: const struct { 74: int line; 75: const char *conn_str;	
	污染路径: 无	
24	缺陷发生位置: /samples/c/demo_topic.c	行号: 210
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 208: 209: if (0) { 210: const char *sqls[] = { 211: "drop topic if exists demo", 212: "drop database if exists foo",	
	污染路径: 无	
25	缺陷发生位置: /tests/c/conformance_test.c	行号: 531
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 529: int r = 0; 530: 531: const char *sqls[] = { 532: "drop table if exists t", 533: "create table t (ts timestamp, bi bigint)",	

	污染路径: 无	
26	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 139
	步骤描述: 数组_cases 定义的没有显式的边界限定	
	代码片段: 137: if (stage != STAGE_CONNECTED) return 0; 138: 139: const struct { 140: const char *sql; 141: int __line__;	
	污染路径: 无	
27	缺陷发生位置: /tests/taos/taos_test.c	行号: 2292
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 2290: int r = 0; 2291: 2292: const char *sqls[] = { 2293: "drop topic if exists topicname", 2294: "drop database if exists tmqdb",	
	污染路径: 无	
28	缺陷发生位置: /src/core/tables.c	行号: 436
	步骤描述: 数组 table_types 定义的没有显式的边界限定	
	代码片段: 434: int r = 0; 435: 436: const char *table_types[] = { 437: "CHILD TABLE", 438: "SUPER TABLE",	
	污染路径: 无	
29	缺陷发生位置: /tests/c/conformance_test.c	行号: 436

	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 434: int r = 0; 435: 436: const char *sqls[] = { 437: "drop table if exists t", 438: "create table t (ts timestamp, bi bigint)",	
	污染路径: 无	
30	缺陷发生位置: /tests/taos/taos_test.c	行号: 1853
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 1851: int r = 0; 1852: 1853: const char *sqls[] = { 1854: "drop database if exists foo", 1855: "create database if not exists foo",	
	污染路径: 无	
31	缺陷发生位置: /tests/c/conformance_test.c	行号: 855
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 853: if (r) return -1; 854: 855: struct { 856: const char *sql; 857: } sqls[] = {	
	污染路径: 无	
32	缺陷发生位置: /tests/taos/taos_regression.c	行号: 143
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 141: static int _prepare_on_connected(TAOS *taos)	

	142: { 143: const sql_t sqls[] = { 144: {"drop database if exists foo", __LINE__}, 145: {"create database if not exists foo", __LINE__},	
	污染路径: 无	
33	缺陷发生位置: /samples/c/demo_topic.c	行号: 64
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 62: 63: if (0) { 64: const char *sqls[] = { 65: "drop topic if exists demo", 66: };	
	污染路径: 无	
34	缺陷发生位置: /tests/cpp/SQLBindCol_ref.cpp	行号: 431
	步骤描述: 数组 exp 定义的没有显式的边界限定	
	代码片段: 429: A(SUCCEEDED(rc), ""); 430: 431: const char *exp[][5] = { 432: {"[2022-09-11 09:57:28.752]", "[name1]", "[20]", "[male]", "[中国人]"}, 433: {"[2022-09-11 09:57:29.753]", "[name2]", "[30]", "[female]", "[苏州人]"},	
	污染路径: 无	
35	缺陷发生位置: /samples/c/demo_topic.c	行号: 71
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 69: 70: if (0) { 71: const char *sqls[] = {	

	72: "drop table if exists demo", 73: "create table demo (ts timestamp, name varchar(20))",	
	污染路径: 无	
36	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 541
	步骤描述: 数组_cases 定义的没有显式的边界限定	
	代码片段: 539: if (stage != STAGE_STATEMENT) return 0; 540: 541: const struct { 542: const char *sql; 543: int __line__; 	
	污染路径: 无	
37	缺陷发生位置: /tests/c/c_test.c	行号: 1043
	步骤描述: 数组 geo 定义的没有显式的边界限定	
	代码片段: 1041: 1042: sql = "select geo from foo.test_types"; 1043: const char geo[] = { 0x01, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf0, 0x3f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40 }; 1044: r = CHECK_WITH_VALUES(handles, 1, sql, 1, 1, geo); 1045: if (r) return -1;	
	污染路径: 无	
38	缺陷发生位置: /tests/c/test_threads.c	行号: 505
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 503: #endif /* } */ 504: 505: sql_t sqls[] = { 506: { "drop topic if exists demo", 0},	

	507: { "drop topic if exists good", 0},	
	污染路径: 无	
39	缺陷发生位置: /tests/c/c_test.c	行号: 1038
	步骤描述: 数组 vb 定义的没有显式的边界限定	
	代码片段: 1036: 1037: sql = "select vb from foo.test_types"; 1038: const char vb[] = { 0x4e, 0xba }; 1039: r = CHECK_WITH_VALUES(handles, 1, sql, 1, 1, vb); 1040: if (r) return -1;	
	污染路径: 无	
40	缺陷发生位置: /samples/c/demo_topic.c	行号: 162
	步骤描述: 数组_topic_demos 定义的没有显式的边界限定	
	代码片段: 160: #define RECORD(x) {x, #x} 161: 162: static struct { 163: topic_demo_f func; 164: const char *name;	
	污染路径: 无	
41	缺陷发生位置: /src/tests/basic.c	行号: 298
	步骤描述: 数组_cases 定义的没有显式的边界限定	
	代码片段: 296: #define OK_INSERT(...) {__LINE__, ##_VA_ARGS__, 1, 0 } 297: #define BAD_INSERT(...) {__LINE__, ##_VA_ARGS__, 0, 0 } 298: static const struct { 299: int line; 300: const char *sql;	

	污染路径: 无	
42	缺陷发生位置: /tests/c/c_test.c	行号: 1495
	步骤描述: 数组 rows 定义的没有显式的边界限定	
	代码片段: 1493: if (r) return -1; 1494: 1495: const char *rows[][2] = { 1496: {"name", "mark"}, 1497: {"测试", "检验"},	
	污染路径: 无	
43	缺陷发生位置: /common/auxiliary.c	行号: 395
	步骤描述: 数组 digits 定义的没有显式的边界限定	
	代码片段: 393: const char* tod_hexify(char *hex, size_t nr, const unsigned char *bin, size_t n) 394: { 395: static const char digits[] = { 396: '0','1','2','3','4','5','6','7','8','9','a','b','c','d','e','f' 397: };	
	污染路径: 无	
44	缺陷发生位置: /taosws/server/taoswsd.c	行号: 317
	步骤描述: 数组 escs 定义的没有显式的边界限定	
	代码片段: 315: 316: const char *matches = "\\t\\f\\r\\n\\b\\\""; 317: const char *escs[] = { 318: "\\\\", 319: "\\t",	
	污染路径: 无	
45	缺陷发生位置:	行号:

	步骤描述: 数组 s_json 定义的没有显式的边界限定	
	代码片段: 849: A(session->current_io->out == NULL, "internal logic error"); 850: 851: const char *s_json = "{\"resp\":{\"errs\":[]}}"; 852: cJSON *json = cJSON_ParseWithLength(s_json, strlen(s_json)); 853: if (!json) return;	
	污染路径: 无	
49	缺陷发生位置:	行号:
	/tests/c/test_threads.c	90
	步骤描述: 数组_tests 定义的没有显式的边界限定	
	代码片段: 88: #define RECORD(x) {x, #x} 89: 90: static struct { 91: test_f func; 92: const char *name;	
50	污染路径: 无	
	缺陷发生位置:	行号:
	/tests/c/c_test.c	1028
	步骤描述: 数组 mark 定义的没有显式的边界限定	
51	代码片段: 1026: 1027: sql = "select mark from foo.test_types"; 1028: const wchar_t mark[] = L"中文 2"; 1029: r = CHECK_WITH_VALUES(handles, 1, sql, 1, 1, mark); 1030: if (r) return -1;	
	污染路径: 无	
	缺陷发生位置:	行号:
	/tests/taos/taos_test.c	2495
	步骤描述: 数组 checkers 定义的没有显式的边界限定	

	代码片段: 2493: if (r) return -1; 2494: 2495: prepare_checker_t checkers[] = { 2496: // subtable insert 2497: {"insert into suzhou using s tags (?) values (?, ?)", 0, 1, 2, 0},	
	污染路径: 无	
52	缺陷发生位置: /src/tests/basic.c	行号: 672
	步骤描述: 数组_cases 定义的没有显式的边界限定	
	代码片段: 670: static int test_sqls_parser(void) 671: { 672: const struct { 673: const char *sqls; 674: const expected_sql_t expects[16];	
	污染路径: 无	
53	缺陷发生位置: /tests/taos/taos_test.c	行号: 2013
	步骤描述: 数组 sqls 定义的没有显式的边界限定	
	代码片段: 2011: int r = 0; 2012: 2013: const char *sqls[] = { 2014: "drop database if exists foo", 2015: "create database if not exists foo",	
	污染路径: 无	
54	缺陷发生位置: /tests/cpp/SQLBindCol_ref.cpp	行号: 174
	步骤描述: 数组 exp 定义的没有显式的边界限定	
	代码片段: 172: } 173:	

	174: const char *exp[][5] = { 175: #ifndef _WIN32 176: {"[2022-09-11 09:57:28.752]", "[name1]", "[20]", "[male]", "[中国人]"}, 污染路径: 无	
	缺陷发生位置: /tests/taos/taos_regression.c	行号: 103
55	步骤描述: 数组_cases 定义的没有显式的边界限定	
	代码片段: 101: 102: #define RECORD(x,y) {x, __LINE__, y} 103: const struct { 104: const char *sql; 105: int __line__; 污染路径: 无	

8 C/C++缺陷类型-不匹配的类型转换

8.1 分项汇总表

序号	C/C++缺陷类型-不匹配的类型转换	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4749	长整数变量赋值给短整数变量必须强制转换	2	严重
2	WK4753	禁止使用无实质作用的类型转换	19	严重

3	WK4794	指针或引用类型的转换中禁止移除 const 或 volatile 属性	12	严重
---	--------	-------------------------------------	----	----

8.2 缺陷/安全漏洞详解

8.2.1 严重类

8.2.1.1 WK4749 长整数变量赋值给短整数变量必须强制转换

缺陷数量：2 个		
缺陷描述： 长整数变量赋值给短整数变量必须强制转换。		
修复意见： 无		
1	缺陷发生位置：	行号：
	/tests/taos/taos_test.c	1061
	步骤描述： 从'unsigned int'到类型'int'的缩小转换	
	代码片段： 1059: 1060: if (ctx->insert) { 1061: if (ctx->subtbl_required && !ctx->tags_described) { 1062: int fieldNum = 0; 1063: TAOS_FIELD_E *fields = NULL;	
2	污染路径：	
	无	
	缺陷发生位置：	行号：
	/tests/taos/taos_test.c	1145
	步骤描述： 从'unsigned int'到类型'int'的缩小转换	
	代码片段： 1143: ejson_t *row0 = r0; 1144: ejson_t *row = r0; 1145: if (ctx->insert && ctx->subtbl_required) {	

	1146: // it's an subtbl insert statement 1147: // we have to split the rows of parameters according to subtbl name
	污染路径: 无

8.2.1.2 WK4753 禁止使用无实质作用的类型转换

缺陷数量：19 个		
缺陷描述： 禁止使用无实质作用的类型转换。		
修复意见： 无		
1	缺陷发生位置：	行号：
	/src/core/stmt.c	6145
	步骤描述： 强制转换为相同类型。	
	代码片段： 6143: 6144: char *inbuf = (char*)s; 6145: char *outbuf = (char*)tsdb_varchar; 6146: 6147: size_t n = CALL_iconv(cnv->cnv, &inbuf, &inbytesleft, &outbuf, &outbytesleft);	
2	污染路径：	
	无	
	缺陷发生位置：	行号：
	/src/core/setup.c	131
	步骤描述： 强制转换为相同类型。	
	代码片段： 129: UINT n = GetDlgItemText(hDlg, pitem->idc, pitem->text, sizeof(pitem->text)); 130: trim_string(pitem->text, n, &pitem->start, &pitem->end); 131: *(char*)pitem->end = '\0'; 132: } 133: }	

	污染路径: 无	
3	缺陷发生位置: /src/core/tables.c	行号: 559
	步骤描述: 强制转换为相同类型.	
	代码片段: 557: while (p < end) { 558: if (!begin) { 559: if (isspace(*p)) { 560: ++p; 561: continue;	
	污染路径: expanded from macro 'isspace' expanded from macro '___isctype'	
4	缺陷发生位置: /src/core/stmt.c	行号: 6297
	步骤描述: 强制转换为相同类型.	
	代码片段: 6295: 6296: char *inbuf = (char*)wstr; 6297: char *outbuf = (char*)tsdb_varchar; 6298: 6299: size_t n = CALL_iconv(cnv->cnv, &inbuf, &inbytesleft, &outbuf, &outbytesleft);	
	污染路径: 无	
5	缺陷发生位置: /tests/taos/taos_test.c	行号: 773
	步骤描述: 强制转换为相同类型.	
	代码片段: 771: is_null += irow; 772: 773: *buffer = (double)d; 774: *length = sizeof(double); 775: *is_null = '\0';	
	污染路径:	

	无	
6	缺陷发生位置： /src/utls/utls.c	行号： 679
	步骤描述： 强制转换为相同类型.	
	代码片段： 677: goto step1; 678: } 679: if (isblank(*p)) { 680: s = ++p; 681: goto step3;	
	污染路径： expanded from macro 'isblank' expanded from macro '___isctype'	
7	缺陷发生位置： /common/auxiliary.c	行号： 389
	步骤描述： 强制转换为相同类型.	
	代码片段： 387: return NULL; 388: } 389: tod_hex2bytes_unsafe(hex, nr, (unsigned char*)buf); 390: return buf; 391: }	
	污染路径： 无	
8	缺陷发生位置： /src/core/stmt.c	行号： 1906
	步骤描述： 强制转换为相同类型.	
	代码片段： 1904: char ox[4096+1]; ox[0] = '\0'; 1905: for (size_t i=0; i<inbytes && i<sizeof(ix)/2; ++i) { 1906: snprintf(ix + i * 2, 3, "%02x", (const unsigned char)inbuf[i]); 1907: } 1908: for (size_t i=0; i<outbytes-outbytesleft && i<sizeof(ox)/2; ++i) {	
	污染路径： 无	

9	缺陷发生位置： /src/utls/utls.c	行号： 846
	步骤描述： 强制转换为相同类型.	
	代码片段： 844: while (p > src) { 845: --p; 846: if (isspace(*(const unsigned char*)p)) continue; 847: *last = p + 1; 848: return;	
	污染路径： expanded from macro 'isspace' expanded from macro '___isctype'	
10	缺陷发生位置： /src/parser/url_parser_tab.c	行号： 246
	步骤描述： 强制转换为相同类型.	
	代码片段： 244: { 245: const unsigned char v = (const unsigned char)c; 246: if (isalnum(v)) return 1; 247: const char *s = "-._~"; 248: return !!strchr(s, c);	
	污染路径： expanded from macro 'isalnum' expanded from macro '___isctype'	
11	缺陷发生位置： /src/utls/utls.c	行号： 833
	步骤描述： 强制转换为相同类型.	
	代码片段： 831: const char *end = src + nr; 832: const char *p = src; 833: while (isspace(*(const unsigned char*)p)) ++p; 834: *first = p; 835: if (p == end) *first = src + nr;	
	污染路径： expanded from macro 'isspace' expanded from macro '___isctype'	
12	缺陷发生位置：	行号：

	/src/core/stmt.c	1909
	步骤描述: 强制转换为相同类型.	
	代码片段: 1907: } 1908: for (size_t i=0; i<outbytes-outbytesleft && i<sizeof(ox)/2; ++i) { 1909: snprintf(ox + i * 2, 3, "%02x", (const unsigned char)outbuf[i]); 1910: } 1911: char buf[16384]; buf[0] = '\0';	
	污染路径: 无	
13	缺陷发生位置: /benchmark/ob.c	行号: 395
	步骤描述: 强制转换为相同类型.	
	代码片段: 393: int32_t i32 = rand(); 394: 395: char *p = (char*)param_bind->buf; 396: SQLLEN *pInd = param_bind->StrLen_or_IndPtr; 397: for (size_t i=0; i<nr_rows; ++i) {	
	污染路径: 无	
14	缺陷发生位置: /src/utils/utils.c	行号: 635
	步骤描述: 强制转换为相同类型.	
	代码片段: 633: 634: step1: 635: if (isblank(*p)) { 636: s = ++p; 637: goto step1;	
	污染路径: expanded from macro 'isblank' expanded from macro '___isctype'	
15	缺陷发生位置: /taosws/server/taoswsd.c	行号: 424

	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 422: static int twc_json_serializer_push_str(twc_json_serializer_t *serializer, const char *s, size_t n) 423: { 424: twc_json_serializer_step_t *step = (twc_json_serializer_step_t*)serializer->get(serializer->user); 425: if (!step) return -1; 426: </pre>	
	污染路径: 无	
16	缺陷发生位置:	行号:
	/src/utls/utls.c	664
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 662: goto step3; 663: } 664: if (isblank(*p)) { 665: if (c != ',') return -1; 666: _check_table_or_stable(s, p-s, table, stable); </pre>	
	污染路径: expanded from macro 'isblank' expanded from macro '___isctype'	
17	缺陷发生位置:	行号:
	/src/parser/url_parser_tab.c	293
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 291: } else { 292: char buf[10]; buf[0] = '\0'; 293: snprintf(buf, sizeof(buf), "%%%02x", (unsigned char)c); 294: r = url_str_append(str, buf, 3); 295: } </pre>	
	污染路径: 无	
18	缺陷发生位置:	行号:

	/taosws/server/taoswsd.c		439
	步骤描述: 强制转换为相同类型.		
	代码片段: <pre> 437: static int twc_json_serializer_push_json(twc_json_serializer_t *serializer, cJSON *json) 438: { 439: twc_json_serializer_step_t *step = (twc_json_serializer_step_t*)serializer->get(serializer->user); 440: if (!step) return -1; 441: </pre>		
	污染路径: 无		
19	缺陷发生位置: /taosws/server/taoswsd.c		行号: 409
	步骤描述: 强制转换为相同类型.		
	代码片段: <pre> 407: static int twc_json_serializer_push_raw_str(twc_json_serializer_t *serializer, const char *s, size_t n) 408: { 409: twc_json_serializer_step_t *step = (twc_json_serializer_step_t*)serializer->get(serializer->user); 410: if (!step) return -1; 411: </pre>		
	污染路径: 无		

8.2.1.3 WK4794 指针或引用类型的转换中禁止移除 const 或

volatile 属性

缺陷数量: 12 个
缺陷描述: 指针或引用类型的转换中禁止移除 const 或 volatile 属性。

修复意见： 无		
1	缺陷发生位置： /src/core/stmt.c	行号： 6296
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段： 6294: size_t outbytesleft = outbytes; 6295: 6296: char *inbuf = (char*)wstr; 6297: char *outbuf = (char*)tsdb_varchar; 6298:	
	污染路径： 无	
2	缺陷发生位置： /src/tests/basic.c	行号： 1515
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段： 1513: do { 1514: char buf[4096]; 1515: char *inbuf = (char*)src; 1516: size_t inbytesleft = src_len; 1517: char *outbuf = buf;	
	污染路径： 无	
3	缺陷发生位置： /src/tests/basic.c	行号： 1273
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段： 1271: char buf[4096]; buf[0] = '\0'; 1272: size_t sz = sizeof(buf); 1273: char *inbuf = (char*)in; 1274: size_t inbytesleft = inlen; 1275: char *outbuf = buf;	
	污染路径：	

	无	
4	缺陷发生位置: /src/utls/utls.c	行号: 160
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 158: } 159: 160: inbuf = (char*)src; 161: inbytesleft = len; 162: outbuf = (char*)mem->base;	
	污染路径: 无	
5	缺陷发生位置: /src/core/tables.c	行号: 521
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 519: 520: char *t = (char*)tables->table_types.base + tables->table_types.nr; 521: char *inbuf = (char*)begin; 522: size_t inbytesleft = p-begin; 523: char *outbuf = t;	
	污染路径: 无	
6	缺陷发生位置: /src/core/env.c	行号: 336
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 334: *pdst = (char*)p; 335: 336: char *inbuf = (char*)src; 337: size_t inbytesleft = nr; 338: char *outbuf = (char*)p;	
	污染路径:	

	无	
7	缺陷发生位置: /src/core/stmt.c	行号: 6144
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 6142: size_t outbytesleft = outbytes; 6143: 6144: char *inbuf = (char*)s; 6145: char *outbuf = (char*)tsdb_varchar; 6146:	
	污染路径: 无	
8	缺陷发生位置: /src/utlis/utlis.c	行号: 235
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 233: 234: while (len) { 235: char *inbuf = (char*)in; 236: size_t inbytesleft = len; 237: char *outbuf = buf;	
	污染路径: 无	
9	缺陷发生位置: /src/tests/basic.c	行号: 1369
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1367: for (size_t i=0; i<times; ++i) { 1368: buf[0] = '\0'; 1369: char *inbuf = (char*)src; 1370: size_t inbytesleft = src_len; 1371: char *outbuf = buf;	
	污染路径: 无	

10	缺陷发生位置： /src/tests/basic.c		行号： 1602
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段： 1600: do { 1601: char buf[4096]; *buf = '\0'; 1602: char *inbuf = (char*)in; 1603: size_t inbytesleft = inlen; 1604: char *outbuf = buf;		
	污染路径： 无		
11	缺陷发生位置： /common/auxiliary.c		行号： 327
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段： 325: if (cnv == (iconv_t)-1) return -1; 326: 327: char *inbuf = (char*)src; 328: char *outbuf = dst; 329: size_t inbytesleft = slen;		
	污染路径： 无		
12	缺陷发生位置： /tests/taos/taosws_regression.c		行号： 102
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.		
	代码片段： 100: } 101: 102: char *inbuf = (char*)src; 103: char *outbuf = dst; 104: size_t inbytesleft = slen;		
	污染路径： 无		

9 C/C++缺陷类型-比较的问题

9.1 分项汇总表

序号	C/C++缺陷类型-比较的问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4759	禁止对指针进行大于或小于的逻辑比较	12	严重

9.2 缺陷/安全漏洞详解

9.2.1 严重类

9.2.1.1 WK4759 禁止对指针进行大于或小于的逻辑比较

缺陷数量：12 个		
缺陷描述： 禁止对指针进行大于或小于的逻辑比较。		
修复意见： 无		
1	缺陷发生位置： /src/utls/utls.c	行号： 466
	步骤描述： 禁止对指针 p 进行大于或小于的逻辑比较	
	代码片段： 464: if (*p == '\\') { 465: if (escaping) return -1; 466: if (p > s) { 467: r = _wild_append(wild, _wild_match_specific, s, p-s); 468: if (r) return -1;	

	污染路径: 无	
2	缺陷发生位置: /src/core/tables.c	行号: 140
	步骤描述: 禁止对指针 p 进行大于或小于的逻辑比较	
	代码片段: 138: 139: const char *p = begin; 140: while (p < end) { 141: // NOTE: non-unicode stored in table_types 142: size_t n = strlen(p);	
	污染路径: 无	
3	缺陷发生位置: /common/ejson_parser_tab.c	行号: 889
	步骤描述: 禁止对指针 curr 进行大于或小于的逻辑比较	
	代码片段: 887: const char *pp = strchr(escape2, c); 888: if (pp) { 889: if (curr > prev) { 890: _SNPRINTF("%.*s", (int)(curr - prev), prev); 891: }	
	污染路径: 无	
4	缺陷发生位置: /src/utls/utls.c	行号: 382
	步骤描述: 禁止对指针 p 进行大于或小于的逻辑比较	
	代码片段: 380: 381: if (inode + 1 == wild->nr) { 382: if (p<end) { 383: DW("nr:%zd;inode:%zd: unmatched", nr, inode); 384: } else {	
	污染路径: 无	
5	缺陷发生位置:	行号:

	/src/utlis/utlis.c	463
	步骤描述: 禁止对指针 p 进行大于或小于的逻辑比较	
	代码片段: 461: const int32_t *p = base; 462: const int32_t *end = (const int32_t*)(wild->ex_ucs4.base + wild->ex_ucs4.nr); 463: while (p < end) { 464: if (*p == '\\') { 465: if (escaping) return -1;	
6	污染路径: 无	
	缺陷发生位置: /src/core/tables.c	行号: 557
	步骤描述: 禁止对指针 p 进行大于或小于的逻辑比较	
7	代码片段: 555: begin = NULL; 556: 557: while (p < end) { 558: if (!begin) { 559: if (isspace(*p)) {	
	污染路径: 无	
	缺陷发生位置: /src/core/stmt.c	行号: 3627
8	步骤描述: 禁止对指针 end 进行大于或小于的逻辑比较	
	代码片段: 3625: const char *start = sqlc_tsdb->tsdb; 3626: const char *end = sqlc_tsdb->tsdb + sqlc_tsdb->tsdb_bytes; 3627: if (end > start && start[0] == '!') { 3628: stmt->tsdb_stmt.is_ext = 1; 3629: } else {	
	污染路径: 无	
8	缺陷发生位置: /src/core/tables.c	行号: 572
	步骤描述:	

	禁止对指针 p 进行大于或小于的逻辑比较	
	代码片段: 570: } 571: 572: if (p > begin) { 573: sr = _tables_add_table_type(tables, TableType, NameLength4, begin, p); 574: if (sr != SQL_SUCCESS) return SQL_ERROR;	
	污染路径: 无	
9	缺陷发生位置: /src/core/stmt.c	行号: 8181
	步骤描述: 禁止对指针 end 进行大于或小于的逻辑比较	
	代码片段: 8179: const char *start = sqlc_tsdb->tsdb; 8180: const char *end = sqlc_tsdb->tsdb + sqlc_tsdb->tsdb_bytes; 8181: if (end > start && start[0] == '!') { 8182: stmt_niy(stmt); 8183: return SQL_ERROR;	
	污染路径: 无	
10	缺陷发生位置: /src/utlis/utlis.c	行号: 387
	步骤描述: 禁止对指针 p 进行大于或小于的逻辑比较	
	代码片段: 385: DW("nr:%zd;inode:%zd: matched", nr, inode); 386: } 387: if (p<end) return; 388: *matched = 1; 389: return;	
	污染路径: 无	
11	缺陷发生位置: /common/ejson_parser_tab.c	行号: 880
	步骤描述: 禁止对指针 curr 进行大于或小于的逻辑比较	
	代码片段:	

	878: const char c = *curr; 879: if (strchr(escape1, c)) { 880: if (curr > prev) { 881: _SPRINTF("%.s", (int)(curr - prev), prev); 882: } 	
	污染路径: 无	
12	缺陷发生位置: /src/utls/utls.c	行号: 356
	步骤描述: 禁止对指针 p 进行大于或小于的逻辑比较	
	代码片段: 354: const int32_t *p = s; 355: const int32_t *end = s + nr; 356: while (p<end && *p) { 357: _wild_exec(wild, p, end-p, inode + 1, matched); 358: if (*matched) DW("nr:%zd;inode:%zd: matched", nr, inode); 	
	污染路径: 无	

10 C/C++ 缺陷类型-字符串问题

10.1 分项汇总表

序号	C/C++缺陷类型-字符串问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK5639	禁止字符串中单独使用 “\”，字符串的终止必须使用\0	1	严重

10.2 缺陷/安全漏洞详解

10.2.1 严重类

10.2.1.1 WK5639 禁止字符串中单独使用 “\”,字符串的终止必须使用\0

缺陷数量： 1 个		
缺陷描述： 只用 “\”并不能控制字符串的终止，应该用 “\0” 控制字符串的终止。		
修复意见： 无		
1	缺陷发生位置： /tests/cpp/test.cpp	行号： 33
	步骤描述： 禁止字符串中单独使用\,字符串的终止必须使用\0	
	代码片段： 31: SQLAllocHandle(SQL_HANDLE_STMT, hdbc, &hstmt); 32: 33: char* create_stable = "CREATE TABLE `meters` (`ts` TIMESTAMP, `current` FLOAT, `voltage` INT, `phase` FLOAT) \ 34: TAGS (`groupid` INT, `location` BINARY(24))"; 35: SQLExecDirect(hstmt, (SQLCHAR*)create_stable, SQL_NTS);	
	污染路径： 无	

11 C/C++缺陷类型-返回值问题

11.1 分项汇总表

序号	C/C++缺陷类型-返回值问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4729	有返回值的函数必须通过返回语句返回	1	严重
2	WK4732	函数返回值的类型必须与定义一致	5	严重
3	WK4733	具有返回值的函数,其返回值如果不被使用,调用是应有void说明	128	严重

11.2 缺陷/安全漏洞详解

11.2.1 严重类

11.2.1.1 WK4729 有返回值的函数必须通过返回语句返回

缺陷数量：1 个		
缺陷描述： 有返回值的函数必须通过返回语句返回。		
修复意见： 无		
1	缺陷发生位置： /src/core/setup.c	行号： 173

	步骤描述: 有返回值的函数必须通过返回语句返回
	代码片段: 171: // } 172: 173: static int GetSelectedComboBoxIndexAndText(HWND hWndCombo, char *buf, size_t sz) 174: { 175: int nCurSel = (int)SendMessage(hWndCombo, CB_GETCURSEL, 0, 0);
	污染路径: 无

11.2.1.2 WK4732 函数返回值的类型必须与定义一致

缺陷数量： 5 个		
缺陷描述: 函数返回值的类型必须与定义一致。		
修复意见: 无		
1	缺陷发生位置: /src/os_port/win_port.c	行号: 91
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'char *'.	
	代码片段: 89: char* tod_getenv(const char *name) 90: { 91: return getenv(name); 92: } 93:	
	污染路径: 无	
2	缺陷发生位置: /tests/cpp/SQLBindCol_ref.cpp	行号: 1704
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'_Bool'，定义类型为'int'.	
	代码片段: 1702: fprintf(stderr, "==%s==\n", r ? "failure" : "success");	

	1703: 1704: return !!r; 1705: } 1706:	
	污染路径: 无	
3	缺陷发生位置: /common/ejson_parser_tab.c	行号: 737
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 735: size_t ejson_obj_count(ejson_t *ejson) 736: { 737: if (!ejson_is_obj(ejson)) return 0; 738: return ejson->_obj.nr; 739: }	
	污染路径: 无	
4	缺陷发生位置: /src/core/ds.c	行号: 195
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'const char *'.	
	代码片段: 193: OA_NIY(ds_conn->conn); 194: 195: return CALL_taos_get_client_info(); 196: } 197:	
	污染路径: 无	
5	缺陷发生位置: /common/ejson_parser_tab.c	行号: 760
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 758: if (!ejson_is_arr(ejson)) { 759: // FIXME: return (size_t)-1; 760: return 0; 761: } 762: return ejson->_arr.nr;	

	污染路径: 无

11.2.1.3 WK4733 具有返回值的函数,其返回值如果不被使用,

调用是应有 void 说明

缺陷数量: 128 个		
缺陷描述: 具有返回值的函数,其返回值如果不被使用,调用时应有 void 说明。		
修复意见: 无		
1	缺陷发生位置: /taosws/server/taoswsd.c	行号: 304
	步骤描述: 不使用返回值的函数'serializer'应使用'void'说明.	
	代码片段: 302: const char *s = (const char*)step->str[0]; 303: size_t n = step->str[1]; 304: serializer->cb(serializer, s, n, 0, serializer->user); 305: return 0; 306: }	
	污染路径: 函数'serializer'的定义点.	
2	缺陷发生位置: /tests/c/odbc_test.c	行号: 88
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 86: } 87: if (StrLen_or_Ind == SQL_NULL_DATA) { 88: E("not implemented yet"); 89: return -1; 90: }	
	污染路径: 函数'E'的定义点.	

3	缺陷发生位置: /src/odbc.c	行号: 108
	步骤描述: 不使用返回值的函数'dladdr'应使用'void'说明.	
	代码片段: 106: #else /* }{ */ 107: DI_info info = {0}; 108: dladdr(_init_image_path_once, &info); 109: snprintf(_global.image_path, sizeof(_global.image_path), "%s", info.dli_fname); 110: tod_basename(_global.image_path, _global.image_name, sizeof(_global.image_name));	
	污染路径: 无	
4	缺陷发生位置: /src/core/conn.c	行号: 140
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	
	代码片段: 138: OA_ILE(prev >= 1); 139: size_t stmts = conn->nr_stmts; 140: OA_ILE(stmts == 0); 141: env_unref(conn->env); 142: conn->env = NULL;	
	污染路径: 函数'OA_ILE'的定义点.	
5	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 98
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 96: iconv_t cnv = iconv_open(tocode, fromcode); 97: if (cnv == (iconv_t)-1) { 98: E("no conversion from `%"s` to `%"s`", fromcode, tocode); 99: return -1; 100: }	
	污染路径: 函数'E'的定义点.	
6	缺陷发生位置:	行号:

	/tests/taos/taos_regression.c	209
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 207: if (_req_tdbname) { 208: E("sql @[%dL]::%s", __line__, _sql); 209: E("expected tblname-parameter, but got ==[none]=="); 210: return -1; 211: }	
	污染路径: 函数'E'的定义点.	
7	缺陷发生位置: /tests/c/sqlite3_test.c	行号: 52
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 50: buf[0] = '\0'; 51: sr = CALL_SQLDriverConnect(hdbc, 0, (SQLCHAR*)conn_str, SQL_NTS, (SQLCHAR*)buf, sizeof(buf), &StringLength2, SQL_DRIVER_NOPROMPT); 52: D("driver completed connection string: [%s]", buf); 53: if (FAILED(sr)) goto fail_connect; 54: }	
	污染路径: 函数'D'的定义点.	
8	缺陷发生位置: /tests/c/edge_cases_test.c	行号: 52
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 50: buf[0] = '\0'; 51: sr = CALL_SQLDriverConnect(hdbc, 0, (SQLCHAR*)conn_str, SQL_NTS, (SQLCHAR*)buf, sizeof(buf), &StringLength2, SQL_DRIVER_NOPROMPT); 52: D("driver completed connection string: [%s]", buf); 53: if (FAILED(sr)) goto fail_connect; 54: }	
	污染路径: 函数'D'的定义点.	

9	缺陷发生位置: /src/core/tsdb.c	行号: 145
	步骤描述: 不使用返回值的函数'TOD_SAFE_FREE'应使用'void'说明.	
	代码片段: 143: paramset->cap = 0; 144: paramset->nr = 0; 145: TOD_SAFE_FREE(paramset->params); 146: } 147:	
	污染路径: 函数'TOD_SAFE_FREE'的定义点.	
10	缺陷发生位置: /src/core/desc.c	行号: 89
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	
	代码片段: 87: int prev = atomic_fetch_sub(&desc->refc, 1); 88: if (prev>1) return desc; 89: OA_ILE(prev==1); 90: 91: _desc_release(desc);	
	污染路径: 函数'OA_ILE'的定义点.	
11	缺陷发生位置: /src/core/setup.c	行号: 108
	步骤描述: 不使用返回值的函数'tod_basename'应使用'void'说明.	
	代码片段: 106: { 107: int ret = GetLastError(); 108: POST_INSTALLER_ERROR(hwndParent, ODBC_ERROR_REQUEST_FAILED, "GetModuleFileName failed, error = %d\n", ret); 109: return -1; 110: }	
	污染路径: expanded from macro 'POST_INSTALLER_ERROR' 函数'tod_basename'的定义点. expanded from macro 'POST_INSTALLER_ERROR'	

12	缺陷发生位置: /src/core/desc.c	行号: 38
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	
	代码片段: 36: desc->conn = conn_ref(conn); 37: int prev = atomic_fetch_add(&conn->descs, 1); 38: OA_ILE(prev >= 0); 39: 40: INIT_TOD_LIST_HEAD(&desc->associated_stmts_as_ARD);	
	污染路径: 函数'OA_ILE'的定义点.	
13	缺陷发生位置: /tests/c/c_test.c	行号: 935
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 933: D("Ordinal Position: %d", ordinal_position); 934: D("Is Nullable: %s", (is_nullable_ind == SQL_NULL_DATA) ? "NULL" : (char*)is_nullable); 935: D("-----"); 936: } while(1); 937:	
	污染路径: 函数'D'的定义点.	
14	缺陷发生位置: /tests/test_helper.c	行号: 154
	步骤描述: 不使用返回值的函数'W'应使用'void'说明.	
	代码片段: 152: if (!val) { 153: char *t1 = cJSON_PrintUnformatted(json); 154: W("no item (%s) found in ==%s==", key, t1); 155: free(t1); 156: return -1;	
	污染路径: 函数'W'的定义点.	
15	缺陷发生位置: /tests/taos/taos_test.c	行号: 101
	步骤描述:	

	不使用返回值的函数'E'应使用'void'说明.	
	代码片段: <pre> 99: buf_serialize[0] = '\0'; 100: ejson_serialize(val, buf_serialize, sizeof(buf_serialize)); 101: E("col #%%d expected null but got ==%s==", i_col, buf_serialize); 102: return -1; 103: }</pre>	
	污染路径: 函数'E'的定义点.	
16	缺陷发生位置:	行号:
	/tests/taos/taos_regression.c	193
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: <pre> 191: if (taos_errno(NULL) == TSDB_CODE_TSC_STMT_TBNAME_ERROR) { 192: if (_errCode == taos_errno(NULL)) return 0; 193: E("sql @[%%dL]::%s", __line__, _sql); 194: E("failed:[0x%x/%%d]%s", taos_errno(NULL), taos_errno(NULL), taos_stmt_errstr(stmt)); 195: return -1;</pre>	
17	污染路径: 函数'E'的定义点.	
	缺陷发生位置:	行号:
	/tests/taos/taosws_regression.c	90
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
18	代码片段: <pre> 88: const char *p = strchr(locale, '.'); 89: p = p ? p + 1 : locale; 90: snprintf(_c_charset, sizeof(_c_charset), "%s", p); 91: #endif 92: }</pre>	
	污染路径: 无	
	缺陷发生位置:	行号:
	/src/core/desc.c	81
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	

	代码片段: 79: OA_ILE(desc); 80: int prev = atomic_fetch_add(&desc->refc, 1); 81: OA_ILE(prev>0); 82: return desc; 83: }	
	污染路径: 函数'OA_ILE'的定义点.	
19	缺陷发生位置: /tests/c/api_test.c	行号: 140
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 138: get_nr_load = dlsym(h1, "get_nr_load"); 139: if (!get_nr_load) { 140: D("`get_nr_load` not exported in [%s]", so); 141: goto fail_dlsym; 142: }	
	污染路径: 函数'D'的定义点.	
20	缺陷发生位置: /src/tests/tls_test.c	行号: 40
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 38: mem_t *mem = tls_get_mem_intermediate(); 39: if (!mem) { 40: E("tls_get_mem_intermediate failed"); 41: return (void*)(uintptr_t)-1; 42: }	
	污染路径: 函数'E'的定义点.	
21	缺陷发生位置: /tests/c/api_test.c	行号: 192
	步骤描述: 不使用返回值的函数'A'应使用'void'说明.	
	代码片段: 190: continue; 191: } 192: A(0, "");	

	193: return -1; 194: }	
	污染路径: 函数'A'的定义点.	
22	缺陷发生位置: /common/auxiliary.c	行号: 94
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: 92: static void _get_local_time(struct timeval *tv0, struct tm *tm0) 93: { 94: gettimeofday(tv0, NULL); 95: time_t t0 = (time_t)tv0->tv_sec; 96: localtime_r(&t0, tm0);	
	污染路径: 函数'gettimeofday'的定义点.	
23	缺陷发生位置: /src/core/setup.c	行号: 164
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 162: char *start, *end; 163: trim_string(s, n, &start, &end); 164: snprintf(buf, sz, "%.s", (int)(end - start), start); 165: } 166:	
	污染路径: 函数'snprintf'的定义点. expanded from macro 'POST_INSTALLER_ERROR'	
24	缺陷发生位置: /tests/c/conformance_test.c	行号: 58
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 56: sr = CALL_SQLConnect(hconn, (SQLCHAR*)dsn, SQL_NTS, (SQLCHAR*)uid, SQL_NTS, (SQLCHAR*)pwd, SQL_NTS); 57: if (FAILED(sr)) { 58: E("connect [dsn:%s,uid:%s,pwd:%s] failed", dsn, uid,	

	pwd); 59: return -1; 60: }	
	污染路径: 函数'E'的定义点.	
25	缺陷发生位置: /src/core/stmt.c	行号: 789
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	
	代码片段: <pre>787: int prev = atomic_fetch_sub(&stmt->refc, 1); 788: if (prev>1) return stmt; 789: OA_ILE(prev==1); 790: 791: tod_list_del(&stmt->node);</pre>	
	污染路径: 函数'OA_ILE'的定义点.	
26	缺陷发生位置: /src/core/env.c	行号: 112
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	
	代码片段: <pre>110: { 111: int prev = atomic_fetch_add(&env->refc, 1); 112: OA_ILE(prev>0); 113: return env; 114: }</pre>	
	污染路径: 函数'OA_ILE'的定义点.	
27	缺陷发生位置: /src/tests/tls_test.c	行号: 82
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: <pre>80: int r = pthread_join(threads[i-1], &p); 81: if (r) { 82: E("pthread_join failed"); 83: ok = 0; 84: }</pre>	

	污染路径: 函数'E'的定义点.	
28	缺陷发生位置: /common/auxiliary.c	行号: 96
	步骤描述: 不使用返回值的函数'localtime_r'应使用'void'说明.	
	代码片段: 94: gettimeofday(tv0, NULL); 95: time_t t0 = (time_t)tv0->tv_sec; 96: localtime_r(&t0, tm0); 97: } 98:	
	污染路径: 无	
29	缺陷发生位置: /tests/test_helper.c	行号: 125
	步骤描述: 不使用返回值的函数'W'应使用'void'说明.	
	代码片段: 123: char *t1 = cJSON_PrintUnformatted(json); 124: char *t2 = cJSON_PrintUnformatted(val); 125: W("item (%s) in ==%s== is not string ==%s==", t1, key, t2); 126: free(t1); 127: free(t2);	
	污染路径: 函数'W'的定义点.	
30	缺陷发生位置: /tests/c/api_test.c	行号: 130
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 128: void *h1 = dlopen(so, RTLD_NOW); 129: if (h1 == NULL) { 130: D("%s", dlerror()); 131: return -1; 132: }	
	污染路径: 函数'D'的定义点.	
31	缺陷发生位置:	行号:

	/tests/test_helper.c	111
	步骤描述: 不使用返回值的函数'W'应使用'void'说明.	
	代码片段: <pre> 109: if (r) { 110: char *t1 = cJSON_PrintUnformatted(json); 111: W("no item (%s) found in ==%s==", key, t1); 112: free(t1); 113: return NULL; </pre>	
32	污染路径: 函数'W'的定义点.	
	缺陷发生位置: /tests/c/api_test.c	行号: 146
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
33	代码片段: <pre> 144: int nr_load = get_nr_load(); 145: if (nr_load != 1) { 146: D("internal logic error in [%s]", so); 147: goto fail_get_nr_load; 148: } </pre>	
	污染路径: 函数'D'的定义点.	
	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 109
34	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: <pre> 107: size_t n = iconv(cnv, &inbuf, &inbytesleft, &outbuf, &outbytesleft); 108: if (n inbytesleft) { 109: E("conversion from ` %s ` to ` %s ` failed", fromcode, tocode); 110: return -1; 111: } </pre>	
	污染路径: 函数'E'的定义点.	
34	缺陷发生位置: /src/tests/tls_test.c	行号: 86
	步骤描述:	

	不使用返回值的函数'E'应使用'void'说明.	
	代码片段: <pre> 84: } 85: if (p) { 86: E("_test_case1_routine failed"); 87: ok = 0; 88: }</pre>	
	污染路径: 函数'E'的定义点.	
35	缺陷发生位置: /src/core/conn.c	行号: 176
	步骤描述: 不使用返回值的函数'OA_Ile'应使用'void'说明.	
	代码片段: <pre> 174: int prev = atomic_fetch_sub(&conn->refc, 1); 175: if (prev>1) return conn; 176: OA_Ile(prev==1); 177: 178: _conn_release(conn);</pre>	
	污染路径: 函数'OA_Ile'的定义点.	
36	缺陷发生位置: /src/odbc.c	行号: 148
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 146: const char *locale = setlocale(LC_CTYPE, ""); 147: if (!locale) return; 148: snprintf(_global.locale_or_ACP, sizeof(_global.locale_or_ACP), "%s", locale); 149: 150: const char *p = strchr(locale, '.');</pre>	
	污染路径: 函数'snprintf'的定义点.	
37	缺陷发生位置: /src/core/conn.c	行号: 168
	步骤描述: 不使用返回值的函数'OA_Ile'应使用'void'说明.	
	代码片段: <pre> 166: {</pre>	

	167: int prev = atomic_fetch_add(&conn->refc, 1); 168: OA_ILE(prev>0); 169: return conn; 170: }	
	污染路径: 函数'OA_ILE'的定义点.	
38	缺陷发生位置: /testes/c/c_test.c	行号: 1333
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 1331: Sleep(0); 1332: #else /* } { */ 1333: sleep(0); 1334: #endif /* } { */ 1335: }	
	污染路径: 无	
39	缺陷发生位置: /testes/c/odbc_test.c	行号: 54
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 52: sr = CALL_SQLConnect(hconn, (SQLCHAR*)dsn, SQL_NTS, (SQLCHAR*)uid, SQL_NTS, (SQLCHAR*)pwd, SQL_NTS); 53: if (FAILED(sr)) { 54: E("connect [dsn:%s,uid:%s,pwd:%s] failed", dsn, uid, pwd); 55: return -1; 56: }	
	污染路径: 函数'E'的定义点.	
40	缺陷发生位置: /src/testes/basic.c	行号: 223
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 221: if (r) { 222: parser_loc_t *loc = ¶m.ctx.bad_token;	

	223: E("parsing[@line:%d]:%s", line, s); 224: E("location:(%d,%d)->(%d,%d)", 225: loc->first_line, loc->first_column, loc->last_line, loc->last_column);	
	污染路径: 函数'E'的定义点.	
41	缺陷发生位置: /tests/c/test_threads.c	行号: 268
	步骤描述: 不使用返回值的函数'W'应使用'void'说明.	
	代码片段: 266: _init_pthread_once(); 267: _init_pthread_once(); 268: W("%zd:%zx:thread:running", tod_get_current_process_id(), tod_get_current_thread_id()); 269: r = _run_with_arg_in_thread((const arg_t*)arg, NULL, 0); 270: W("%zd:%zx:thread:return:%d", tod_get_current_process_id(), tod_get_current_thread_id(), r);	
	污染路径: 函数'W'的定义点.	
42	缺陷发生位置: /tests/c/edge_cases_test.c	行号: 565
	步骤描述: 不使用返回值的函数'CHECK'应使用'void'说明.	
	代码片段: 563: srand((unsigned int)time(0)); 564: 565: CHECK(!test_connect(NULL, "xTAOS_ODBC_DSN", NULL, NULL)); 566: 567: #if 0 /* { */	
	污染路径: 函数'CHECK'的定义点.	
43	缺陷发生位置: /src/core/errs.c	行号: 61
	步骤描述: 不使用返回值的函数'OD'应使用'void'说明.	
	代码片段: 59: err_t *err = (err_t*)calloc(1, sizeof(*err)); 60: if (!err) {	

	<pre> 61: OD("out of memory"); 62: return; 63: }</pre>	
	污染路径: 函数'OD'的定义点.	
44	缺陷发生位置: /common/logger.c	行号: 253
	步骤描述: 不使用返回值的函数'pthread_once'应使用'void'说明.	
	代码片段: <pre> 251: if (!init) { 252: static pthread_once_t once; 253: pthread_once(&once, _init_all); 254: init = 1; 255: }</pre>	
	污染路径: 函数'pthread_once'的定义点.	
45	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 169
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: <pre> 167: E("failed:[%d]%s", e, ws_errstr(res)); 168: } else { 169: E("succeed unexpectedly"); 170: } 171: r = -1;</pre>	
	污染路径: 函数'E'的定义点.	
46	缺陷发生位置: /src/core/conn.c	行号: 116
	步骤描述: 不使用返回值的函数'OA_Ile'应使用'void'说明.	
	代码片段: <pre> 114: conn->env = env_ref(env); 115: int prev = atomic_fetch_add(&env->conns, 1); 116: OA_Ile(prev >= 0); 117: 118: errs_init(&conn->errs);</pre>	

	污染路径: 函数'OA_ILE'的定义点.	
47	缺陷发生位置: /common/ejson_parser_tab.c	行号: 588
	步骤描述: 不使用返回值的函数'sscanf'应使用'void'说明.	
	代码片段: 586: snprintf(buf, sizeof(buf), "%.s", (int)n, s); 587: double dbl = 0; 588: sscanf(buf, "%lg", &dbl); 589: return ejson_new_num(dbl); 590: }	
	污染路径: 函数'sscanf'的定义点.	
48	缺陷发生位置: /tests/c/c_test.c	行号: 1339
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 1337: for (size_t i=0; i<ds->_nr_cases; ++i) { 1338: char sql[4096]; sql[0] = '\0'; 1339: snprintf(sql, sizeof(sql), "insert into foobar.demo (ts, name, mark) values (now()+%zds, '%s', '%s')", i, ds->_cases[i] [0], ds->_cases[i][1]); 1340: sr = CALL_SQLExecDirect(handles.hstmt, (SQLCHAR*)sql, SQL_NTS); 1341: CALL_SQLCloseCursor(handles.hstmt);	
	污染路径: 函数'snprintf'的定义点.	
49	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 675
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: 673: int r = 0; 674: 675: PROFILE(r = _do_run(hstmt, cfg)); 676: 677: return r ? -1 : 0;	
	污染路径:	

	expanded from macro 'PROFILE'	
50	缺陷发生位置: /tests/c/conformance_test.c	行号: 71
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: <pre> 69: sr = CALL_SQLDriverConnect(hconn, NULL, (SQLCHAR*)connstr, SQL_NTS, NULL, 0, NULL, SQL_DRIVER_NOPROMPT); 70: if (FAILED(sr)) { 71: E("driver_connect [connstr:%s] failed", connstr); 72: return -1; 73: }</pre>	
	污染路径: 函数'E'的定义点.	
51	缺陷发生位置: /tests/taos/taos_test.c	行号: 278
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: <pre> 276: ejson_t *val = ejson_arr_get(row, i); 277: if (!val) { 278: E("col #%d: lack of corresponding data", i+1); 279: return -1; 280: }</pre>	
	污染路径: 函数'E'的定义点.	
52	缺陷发生位置: /src/parser/url_parser_tab.c	行号: 51
	步骤描述: 不使用返回值的函数'TOD_SAFE_FREE'应使用'void'说明.	
	代码片段: <pre> 49: { 50: if (!str) return; 51: TOD_SAFE_FREE(str->str); 52: str->cap = 0; 53: str->nr = 0;</pre>	
	污染路径: 函数'TOD_SAFE_FREE'的定义点.	
53	缺陷发生位置:	行号:

	/common/ejson_parser_tab.c	586
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 584: { 585: char buf[128]; 586: snprintf(buf, sizeof(buf), "%.s", (int)n, s); 587: double dbl = 0; 588: sscanf(buf, "%lg", &dbl);	
54	污染路径: 函数'snprintf'的定义点.	
	缺陷发生位置: /src/os_port/win_port.c	行号: 221
	步骤描述: 不使用返回值的函数'strerror_s'应使用'void'说明.	
55	代码片段: 219: char *tod_strerror_x(int errnum, char *buf, size_t buflen) 220: { 221: strerror_s(buf, buflen, errnum); 222: return buf; 223: }	
	污染路径: 函数'strerror_s'的定义点.	
	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 286
56	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 284: for (size_t i=0; i<rows; ++i) { 285: int v = rand(); 286: snprintf(ps + i * (len + 1), len + 1, "%0*d", (int)len, v); 287: p[i] = len; 288: // E("char:[%s]", ps + i * (len + 1));	
	污染路径: 无	
56	缺陷发生位置: /tests/c/c_test.c	行号: 1300
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段:	

	1298: if (sr != SQL_SUCCESS) return -1; 1299: // FIXME: vulnerability 1300: snprintf(drop_buf, sizeof(drop_buf), "drop topic `%s`", topic_name_buf); 1301: CALL_SQLCloseCursor(handles->hstmt); 1302: CALL_SQLFreeStmt(handles->hstmt, SQL_UNBIND);	
	污染路径: 函数'snprintf'的定义点.	
57	缺陷发生位置: /testes/c/sqlite3_test.c	行号: 304
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 302: 303: if (nr != batches * ARRAY_SIZE) { 304: D("internal logic error"); 305: r = -1; 306: }	
	污染路径: 函数'D'的定义点.	
58	缺陷发生位置: /src/core/stmt.c	行号: 781
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	
	代码片段: 779: { 780: int prev = atomic_fetch_add(&stmt->refc, 1); 781: OA_ILE(prev>0); 782: return stmt; 783: }	
	污染路径: 函数'OA_ILE'的定义点.	
59	缺陷发生位置: /src/core/setup.c	行号: 102
	步骤描述: 不使用返回值的函数'tod_basename'应使用'void'说明.	
	代码片段: 100: { 101: int ret = GetLastError(); 102: POST_INSTALLER_ERROR(hwndParent,	

	ODBC_ERROR_REQUEST_FAILED, "GetModuleHandle failed, error = %d\n", ret); 103: return -1; 104: }	
	污染路径: expanded from macro 'POST_INSTALLER_ERROR' 函数'tod_basename'的定义点. expanded from macro 'POST_INSTALLER_ERROR'	
60	缺陷发生位置: /tests/c/conformance_test.c	行号: 133
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 131: if (ind == SQL_NULL_DATA) { 132: if (exp) { 133: E("(r#%d, c#%d) [%s] expected, but got ==%s==", irow+1, icol+1, exp, varchar); 134: return -1; 135: }	
	污染路径: 函数'E'的定义点.	
61	缺陷发生位置: /tests/c/edge_cases_test.c	行号: 271
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 269: 270: if (nr != batches * ARRAY_SIZE) { 271: D("internal logic error"); 272: r = -1; 273: }	
	污染路径: 函数'D'的定义点.	
62	缺陷发生位置: /tests/taos/taosws_regression.c	行号: 165
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 163: int e = ws_errno(res); 164: if ((!!e) ^ (!expected_ok)) {	

	165: E("executing sql @[%dL]:%s", __line__, sql); 166: if (expected_ok) { 167: E("failed:[%d]%s", e, ws_errstr(res));	
	污染路径: 函数'E'的定义点.	
63	缺陷发生位置: /tests/c/edge_cases_test.c	行号: 307
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 305: 306: if (nr != batches * ARRAY_SIZE) { 307: D("internal logic error"); 308: r = -1; 309: }	
	污染路径: 函数'D'的定义点.	
64	缺陷发生位置: /common/ejson_parser_tab.c	行号: 191
	步骤描述: 不使用返回值的函数'tod_hex2bytes_unsafe'应使用'void'说明.	
	代码片段: 189: size_t hn = sizeof(buf) * 2; 190: if (hn > n) hn = n; 191: tod_hex2bytes_unsafe(v, hn, buf); 192: int r = _ejson_bin_append(bin, buf, hn / 2); 193: if (r) return -1;	
	污染路径: 函数'tod_hex2bytes_unsafe'的定义点.	
65	缺陷发生位置: /src/core/stmt.c	行号: 1906
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 1904: char ox[4096+1]; ox[0] = '\0'; 1905: for (size_t i=0; i<inbytes && i<sizeof(ix)/2; ++i) { 1906: snprintf(ix + i * 2, 3, "%02x", (const unsigned char)inbuf[i]); 1907: } 1908: for (size_t i=0; i<outbytes-outbytesleft &&	

	i<sizeof(ox)/2; ++i) {	
	污染路径: 函数'snprintf'的定义点.	
66	缺陷发生位置: /samples/c/demo_topic.c	行号: 198
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 196: sr = CALL_SQLDriverConnect(hconn, NULL, (SQLCHAR*)connstr, SQL_NTS, NULL, 0, NULL, SQL_DRIVER_NOPROMPT); 197: if (FAILED(sr)) { 198: E("driver_connect [connstr:%s] failed", connstr); 199: return -1; 200: }	
	污染路径: 函数'E'的定义点.	
67	缺陷发生位置: /src/tests/basic.c	行号: 63
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 61: { 62: const char *s = tod_get_locale_or_ACP(); 63: D("locale_or_ACP: [%s]", s); 64: return 0; 65: }	
	污染路径: 函数'D'的定义点.	
68	缺陷发生位置: /tests/c/c_test.c	行号: 452
	步骤描述: 不使用返回值的函数'tod_get_format_current_local_timestamp_ms'应使用'void'说明.	
	代码片段: 450: r = execute_with_params(connstr, sql, 3, "2023-05-07 10:11:44.215", "测试", "人物 y"); 451: if (r) return -1; 452: tod_get_format_current_local_timestamp_ms(tmbuf, sizeof(tmbuf)); 453: DUMP("current:%s", tmbuf);	

	454: r = execute_with_params(connstr, sql, 3, tmbuf, "测试", "人物 z");	
	污染路径: 函数'tod_get_format_current_local_timestamp_ms'的定义点.	
69	缺陷发生位置: /src/core/stmt.c	行号: 1912
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 1910: } 1911: char buf[16384]; buf[0] = '\0'; 1912: snprintf(buf, sizeof(buf), "%s=>%s:%zd,%zd;%zd,%zd;0x%.*s;0x%.*s", 1913: fromcode, tocode, inbytes, inbytesleft, outbytes, outbytesleft, 1914: (int)(inbytes*2), ix,	
	污染路径: 函数'snprintf'的定义点.	
70	缺陷发生位置: /tests/c/api_test.c	行号: 300
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 298: static int do_sql_stmt_execute_direct(SQLHANDLE stmth) 299: { 300: CHK2(test_sql_stmt_execute_direct, stmth, "xshow databases", -1); 301: CHK2(test_sql_stmt_execute_direct, stmth, "show databases", 0); 302: CHK2(test_sql_stmt_execute_direct, stmth, "drop database if exists foo", 0);	
	污染路径: expanded from macro 'CHK2' expanded from macro 'TEST_CASE_BEG' 函数'D'的定义点.	
71	缺陷发生位置: /src/core/tsdb.c	行号: 67
	步骤描述: 不使用返回值的函数'OA_NIY'应使用'void'说明.	
	代码片段:	

	65: 66: if (tt <= 0 && xfraction < 0) { 67: OA_NIY(0); 68: } 69:	
	污染路径: 函数'OA_NIY'的定义点.	
72	缺陷发生位置: /tests/cpp/test.cpp	行号: 54
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 52: // read data from table 53: char select_sql[256]; 54: sprintf(select_sql, "select ts, current, voltage from d0"); 55: SQLExecDirect(hstmt, (SQLCHAR*)select_sql, SQL_NTS); 56:	
	污染路径: 无	
73	缺陷发生位置: /tests/c/conformance_test.c	行号: 139
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 137: } 138: if(ind < 0) { 139: E("not implemented yet"); 140: return -1; 141: }	
	污染路径: 函数'E'的定义点.	
74	缺陷发生位置: /common/logger.c	行号: 304
	步骤描述: 不使用返回值的函数'vsprintf'应使用'void'说明.	
	代码片段: 302: va_list ap; 303: va_start(ap, fmt); 304: vsnprintf(p, l, fmt, ap); 305: va_end(ap);	

	306:	
	污染路径: 无	
75	缺陷发生位置: /src/tests/basic.c	行号: 230
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 228: } 229: if (cmp_strs(expected->driver, param.conn_cfg->driver)) { 230: E("parsing[@line:%d]:%s", line, s); 231: E("driver expected to be `'%s`', but got ==%s==", expected->driver, param.conn_cfg->driver); 232: r = -1;	
	污染路径: 函数'E'的定义点.	
76	缺陷发生位置: /src/core/conn.c	行号: 138
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	
	代码片段: 136: 137: int prev = atomic_fetch_sub(&conn->env->conns, 1); 138: OA_ILE(prev >= 1); 139: size_t stmts = conn->nr_stmts; 140: OA_ILE(stmts == 0);	
	污染路径: 函数'OA_ILE'的定义点.	
77	缺陷发生位置: /tests/taos/taos_regression.c	行号: 208
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 206: 207: if (_req_tname) { 208: E("sql @[:%dL]:%s", __line__, _sql); 209: E("expected tblname-parameter, but got ==[none]=="); 210: return -1;	

	污染路径: 函数'E'的定义点.	
78	缺陷发生位置: /tests/test_helper.c	行号: 118
	步骤描述: 不使用返回值的函数'W'应使用'void'说明.	
	代码片段: <pre> 116: if (!val) { 117: char *t1 = cJSON_PrintUnformatted(json); 118: W("no item (%s) found in ==%s==", key, t1); 119: free(t1); 120: return NULL; </pre>	
	污染路径: 函数'W'的定义点.	
79	缺陷发生位置: /src/core/setup.c	行号: 102
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 100: { 101: int ret = GetLastError(); 102: POST_INSTALLER_ERROR(hwndParent, ODBC_ERROR_REQUEST_FAILED, "GetModuleHandle failed, error = %d\n", ret); 103: return -1; 104: } </pre>	
	污染路径: expanded from macro 'POST_INSTALLER_ERROR' 函数'snprintf'的定义点. expanded from macro 'POST_INSTALLER_ERROR'	
80	缺陷发生位置: /src/core/env.c	行号: 41
	步骤描述: 不使用返回值的函数'CALL_taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 39: // need to check later 40: #else 41: CALL_taos_cleanup(); 42: #endif 43: } </pre>	

	污染路径: 函数'CALL_taos_cleanup'的定义点.	
81	缺陷发生位置: /src/utlis/utlis.c	行号: 888
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 886: const char *start, *end; 887: trim_string(src, len, &start, &end); 888: snprintf(dst, n, "%.s", (int)(end-start), start); 889: } 890:	
	污染路径: 无	
82	缺陷发生位置: /src/core/stmt.c	行号: 1916
	步骤描述: 不使用返回值的函数'OW'应使用'void'说明.	
	代码片段: 1914: (int)(inbytes*2), ix, 1915: (int)(outbytes-outbytesleft)*2, ox); 1916: OW("%s\n", buf); 1917: } 1918:	
	污染路径: 函数'OW'的定义点.	
83	缺陷发生位置: /tests/c/test_threads.c	行号: 131
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 129: sr = CALL_SQLDriverConnect(hconn, NULL, (SQLCHAR*)connstr, SQL_NTS, NULL, 0, NULL, SQL_DRIVER_NOPROMPT); 130: if (FAILED(sr)) { 131: E("driver_connect [connstr:%s] failed", connstr); 132: return -1; 133: }	
	污染路径:	

	函数'E'的定义点.	
84	缺陷发生位置: /benchmark/tb.c	行号: 308
	步骤描述: 不使用返回值的函数'CALL_taos_init'应使用'void'说明.	
	代码片段: 306: handles_t handles = {0}; 307: 308: CALL_taos_init(); 309: 310: r = run(&handles, argc, argv);	
	污染路径: 函数'CALL_taos_init'的定义点.	
85	缺陷发生位置: /src/tests/tls_test.c	行号: 46
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 44: int r = mem_keep(mem, sz); 45: if (r) { 46: E("mem_keep failed"); 47: return (void*)(uintptr_t)-2; 48: }	
	污染路径: 函数'E'的定义点.	
86	缺陷发生位置: /src/tests/basic.c	行号: 236
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 234: } 235: if (cmp_strs(expected->url, param.conn_cfg->url)) { 236: E("parsing[@line:%d]:%s", line, s); 237: E("url expected to be `'%s`', but got ==%s==", expected->url, param.conn_cfg->url); 238: r = -1;	
	污染路径: 函数'E'的定义点.	
87	缺陷发生位置: /tests/c/sqlite3_test.c	行号: 513

	步骤描述: 不使用返回值的函数'CHECK'应使用'void'说明.	
	代码片段: 511: static int test(void) 512: { 513: CHECK(!test_connect("Driver={SQLite3};Database=/tmp/bar.sqlite3", NULL, NULL, NULL)); 514: 515: SQLRETURN sr;	
	污染路径: 函数'CHECK'的定义点.	
88	缺陷发生位置:	行号:
	/taosws/server/taoswsd.c	362
	步骤描述: 不使用返回值的函数'serializer'应使用'void'说明. 代码片段: 360: } break; 361: case cJSON_True: { 362: serializer->cb(serializer, "true", 4, 0, serializer->user); 363: } break; 364: case cJSON_NULL: {	
	污染路径: 函数'serializer'的定义点.	
89	缺陷发生位置:	行号:
	/src/core/env.c	120
	步骤描述: 不使用返回值的函数'OA_Ile'应使用'void'说明. 代码片段: 118: int prev = atomic_fetch_sub(&env->refc, 1); 119: if (prev>1) return env; 120: OA_Ile(prev==1); 121: 122: _env_release(env);	
	污染路径: 函数'OA_Ile'的定义点.	
90	缺陷发生位置:	行号:
	/tests/c/odbc_test.c	101
	步骤描述:	

	不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 99: double dr = 0; 100: ejson_num_get(val, &dr); 101: char lbuf[64]; snprintf(lbuf, sizeof(lbuf), "%lg", dl); 102: char rbuf[64]; snprintf(rbuf, sizeof(rbuf), "%lg", dr); 103: if (strcmp(lbuf, rbuf) == 0) return 0; </pre>	
	污染路径: 函数'snprintf'的定义点.	
91	缺陷发生位置: /tests/c/conformance_test.c	行号: 144
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: <pre> 142: 143: if (!exp) { 144: E("(r#%d, c#%d) `null` expected, but got ==%s==", irow+1, icol+1, varchar); 145: return -1; 146: } </pre>	
92	污染路径: 函数'E'的定义点.	
	缺陷发生位置: /tests/c/odbc_test.c	行号: 67
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
93	代码片段: <pre> 65: sr = CALL_SQLDriverConnect(hconn, NULL, (SQLCHAR*)connstr, SQL_NTS, NULL, 0, NULL, SQL_DRIVER_NOPROMPT); 66: if (FAILED(sr)) { 67: E("driver_connect [connstr:%s] failed", connstr); 68: return -1; 69: } </pre>	
	污染路径: 函数'E'的定义点.	
	缺陷发生位置: /src/parser/url_parser_tab.c	行号: 94
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	

	代码片段: 92: v = (c - 'A' + 10) << (4*(1-i)); 93: } else { 94: OA_ILE(0); 95: } 96: }	
	污染路径: 函数'OA_ILE'的定义点.	
94	缺陷发生位置: /src/core/env.c	行号: 89
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	
	代码片段: 87: { 88: int conns = atomic_load(&env->conns); 89: OA_ILE(conns == 0); 90: errs_release(&env->errs); 91: mem_release(&env->mem);	
	污染路径: 函数'OA_ILE'的定义点.	
95	缺陷发生位置: /tests/taos/taos_test.c	行号: 271
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 269: buf_serialize[0] = '\0'; 270: ejson_serialize(row, buf_serialize, sizeof(buf_serialize)); 271: E("json array is required but got ==%s==", buf_serialize); 272: return -1; 273: }	
	污染路径: 函数'E'的定义点.	
96	缺陷发生位置: /src/tests/basic.c	行号: 242
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 240: }	

	<pre> 241: if (cmp_strs(expected->dsn, param.conn_cfg->dsn)) { 242: E("parsing[@line:%d]:%s", line, s); 243: E("dsn expected to be `'%s`", but got ==%s==", expected->dsn, param.conn_cfg->dsn); 244: r = -1; </pre>	
	污染路径: 函数'E'的定义点.	
97	缺陷发生位置: /benchmark/tb.c	行号: 314
	步骤描述: 不使用返回值的函数'CALL_taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 312: handles_release(&handles); 313: 314: CALL_taos_cleanup(); 315: 316: DUMP("-=%s=-", r ? "failure" : "success"); </pre>	
	污染路径: 函数'CALL_taos_cleanup'的定义点.	
98	缺陷发生位置: /taosws/server/taoswsd.c	行号: 332
	步骤描述: 不使用返回值的函数'serializer'应使用'void'说明.	
	代码片段: <pre> 330: if (twc_json_serializer_push_str(serializer, s+i+1, n-i-1)) return -1; 331: if (twc_json_serializer_push_raw_str(serializer, esc, strlen(esc))) return -1; 332: serializer->cb(serializer, s, i, 0, serializer->user); 333: return 0; 334: } </pre>	
	污染路径: 函数'serializer'的定义点.	
99	缺陷发生位置: /src/parser/url_parser_tab.c	行号: 140
	步骤描述: 不使用返回值的函数'TOD_SAFE_FREE'应使用'void'说明.	
	代码片段: <pre> 138: TOD_SAFE_FREE(strs->strs[i]); </pre>	

	139: } 140: TOD_SAFE_FREE(strs->strs); 141: strs->cap = 0; 142: strs->nr = 0;	
	污染路径: 函数'TOD_SAFE_FREE'的定义点.	
100	缺陷发生位置: /tests/taos/taos_test.c	行号: 64
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 62: char **arr = (char**)realloc(buffers->arr, cap * sizeof(*arr)); 63: if (!arr) { 64: E("out of memroy"); 65: return -1; 66: }	
	污染路径: 函数'E'的定义点.	
101	缺陷发生位置: /tests/taos/taos_test.c	行号: 124
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 122: char *p = strdup(base, len); 123: if (!p) { 124: E("out of memory"); 125: return -1; 126: }	
	污染路径: 函数'E'的定义点.	
102	缺陷发生位置: /tests/c/test_threads.c	行号: 270
	步骤描述: 不使用返回值的函数'W'应使用'void'说明.	
	代码片段: 268: W("%zd:%zx:thread:running", tod_get_current_process_id(), tod_get_current_thread_id()); 269: r = _run_with_arg_in_thread((const arg_t*)arg, NULL, 0); 270: W("%zd:%zx:thread:return:%d",	

	tod_get_current_process_id(), tod_get_current_thread_id(), r); 271: if (r) return -1; 272: return 0;	
	污染路径: 函数'W'的定义点.	
103	缺陷发生位置: /src/core/tsdb.c	行号: 120
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	
	代码片段: 118: w, fraction); 119: 120: OA_ILE(n > 0); 121: 122: return n;	
	污染路径: 函数'OA_ILE'的定义点.	
104	缺陷发生位置: /tests/c/test_threads.c	行号: 253
	步骤描述: 不使用返回值的函数'A'应使用'void'说明.	
	代码片段: 251: static void _init_tick(void) 252: { 253: A(_tick == 0, ""); 254: ++_tick; 255: }	
	污染路径: 函数'A'的定义点.	
105	缺陷发生位置: /src/core/setup.c	行号: 108
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 106: { 107: int ret = GetLastError(); 108: POST_INSTALLER_ERROR(hwndParent, ODBC_ERROR_REQUEST_FAILED, "GetModuleFileName failed, error = %d\n", ret); 109: return -1;	

	110: }	
	污染路径: expanded from macro 'POST_INSTALLER_ERROR' 函数'snprintf'的定义点. expanded from macro 'POST_INSTALLER_ERROR'	
106	缺陷发生位置: /benchmark/taos_benchmark.c	行号: 602
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: 600: if (r) return -1; 601: 602: PROFILE(r = do_benchmark_case0(stmt, cfg)); 603: 604: return r ? -1 : 0;	
	污染路径: expanded from macro 'PROFILE'	
107	缺陷发生位置: /common/odbc_helpers.c	行号: 126
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 124: sr = CALL_SQLDriverConnect(hconn, NULL, (SQLCHAR*)connstr, SQL_NTS, (SQLCHAR*)buf, sizeof(buf), &StringLength, SQL_DRIVER_COMPLETE); 125: if (FAILED(sr)) { 126: E("driver_connect [%s] failed", connstr); 127: return -1; 128: }	
	污染路径: 函数'E'的定义点.	
108	缺陷发生位置: /src/core/env.c	行号: 222
	步骤描述: 不使用返回值的函数'OA_NIY'应使用'void'说明.	
	代码片段: 220: 221: env_append_err_format(env, "01S02", 0, "Optional value changed: `%(0x%x/%d)` is substituted by default", sql_env_attr(Attribute), Attribute, Attribute);	

	222: OA_NIY(0); 223: return SQL_SUCCESS_WITH_INFO; 224: }	
	污染路径: 函数'OA_NIY'的定义点.	
109	缺陷发生位置: /tests/taos/taos_regression.c	行号: 233
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 231: } 232: if (_errCode == taos_errno(NULL)) return 0; 233: E("sql @[%dL]::%s", __line__, _sql); 234: E("failed:[0x%x/%d]%s", taos_errno(NULL), taos_errno(NULL), taos_stmt_errstr(stmt)); 235: return -1;	
	污染路径: 函数'E'的定义点.	
110	缺陷发生位置: /common/auxiliary.c	行号: 77
	步骤描述: 不使用返回值的函数'gmtime_r'应使用'void'说明.	
	代码片段: 75: time_t t0 = 12*60*60; 76: struct tm tm; 77: gmtime_r(&t0, &tm); 78: _local_timezone = t0 - mktime(&tm); 79: }	
	污染路径: 无	
111	缺陷发生位置: /src/parser/url_parser_tab.c	行号: 293
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 291: } else { 292: char buf[10]; buf[0] = '\0'; 293: snprintf(buf, sizeof(buf), "%%%02x", (unsigned char)c); 294: r = url_str_append(str, buf, 3);	

	295: }	
	污染路径: 函数'snprintf'的定义点.	
112	缺陷发生位置: /tests/taos/taos_regression.c	行号: 201
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 199: if (r) { 200: if (_errCode == taos_errno(NULL)) return 0; 201: E("sql @[%dL]::%s", __line__, _sql); 202: E("failed:[0x%x/%d]%s", taos_errno(NULL), taos_errno(NULL), taos_stmt_errstr(stmt)); 203: return -1;	
	污染路径: 函数'E'的定义点.	
113	缺陷发生位置: /taosws/server/taoswsd.c	行号: 335
	步骤描述: 不使用返回值的函数'serializer'应使用'void'说明.	
	代码片段: 333: return 0; 334: } 335: serializer->cb(serializer, s, n, 0, serializer->user); 336: return 0; 337: }	
	污染路径: 函数'serializer'的定义点.	
114	缺陷发生位置: /src/tests/tls_test.c	行号: 53
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 51: mem_t *mem2 = tls_get_mem_intermediate(); 52: if (mem != mem2) { 53: E("tls_get_mem_intermediate implementation failure"); 54: return (void*)(uintptr_t)-3; 55: }	
	污染路径:	

	函数'E'的定义点.	
115	缺陷发生位置: /tests/c/test_threads.c	行号: 118
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 116: sr = CALL_SQLConnect(hconn, (SQLCHAR*)dsn, SQL_NTS, (SQLCHAR*)uid, SQL_NTS, (SQLCHAR*)pwd, SQL_NTS); 117: if (FAILED(sr)) { 118: E("connect [dsn:%s,uid:%s,pwd:%s] failed", dsn, uid, pwd); 119: return -1; 120: }	
	污染路径: 函数'E'的定义点.	
116	缺陷发生位置: /src/core/tsdb.c	行号: 1240
	步骤描述: 不使用返回值的函数'OA_NIY'应使用'void'说明.	
	代码片段: 1238: } 1239: #endif /*] */ 1240: OA_NIY(r == 0); 1241: stmt->stmt = NULL; 1242: }	
	污染路径: 函数'OA_NIY'的定义点.	
117	缺陷发生位置: /tests/cpp/test.cpp	行号: 43
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 41: // write data into test table 42: char insert_sql[256]; 43: sprintf(insert_sql, "INSERT INTO `d0` values(now - 10s, 10, 116, 0.32)"); 44: SQLExecDirect(hstmt, (SQLCHAR*)insert_sql, SQL_NTS); 45: SQLLEN numberOfrows;	
	污染路径:	

	无	
118	缺陷发生位置: /tests/c/odbc_test.c	行号: 84
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 82: if (FAILED(sr)) return -1; 83: if (StrLen_or_Ind == SQL_NO_TOTAL) { 84: E("not implemented yet"); 85: return -1; 86: }	
	污染路径: 函数'E'的定义点.	
119	缺陷发生位置: /tests/c/sqlite3_test.c	行号: 268
	步骤描述: 不使用返回值的函数'D'应使用'void'说明.	
	代码片段: 266: 267: if (nr != batches * ARRAY_SIZE) { 268: D("internal logic error"); 269: r = -1; 270: }	
	污染路径: 函数'D'的定义点.	
120	缺陷发生位置: /src/core/desc.c	行号: 52
	步骤描述: 不使用返回值的函数'OA_ILE'应使用'void'说明.	
	代码片段: 50: { 51: int prev = atomic_fetch_sub(&desc->conn->descs, 1); 52: OA_ILE(prev >= 1); 53: 54: descriptor_release(&desc->descriptor);	
	污染路径: 函数'OA_ILE'的定义点.	
121	缺陷发生位置: /taosws/server/taoswsd.c	行号: 359
	步骤描述:	

	不使用返回值的函数'serializer'应使用'void'说明.	
	代码片段: <pre> 357: switch (type) { 358: case cJSON_False: { 359: serializer->cb(serializer, "false", 5, 0, serializer->user); 360: } break; 361: case cJSON_True: { </pre>	
	污染路径: 函数'serializer'的定义点.	
122	缺陷发生位置: /src/core/charset.c	行号: 111
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 109: { 110: char key[256]; // NOTE: big enough? 111: snprintf(key, sizeof(key), "%s,%s", fromcode, tocode); 112: void *val = NULL; 113: hash_table_get(mgr->convs, key, &val); </pre>	
	污染路径: 函数'snprintf'的定义点.	
123	缺陷发生位置: /common/ejson_parser_tab.c	行号: 312
	步骤描述: 不使用返回值的函数'tod_hex2bytes_unsafe'应使用'void'说明.	
	代码片段: <pre> 310: size_t len = strlen(utf16be); 311: 312: tod_hex2bytes_unsafe(utf16be, len, (unsigned char*)buf); 313: 314: char *inbuf = buf; </pre>	
	污染路径: 函数'tod_hex2bytes_unsafe'的定义点.	
124	缺陷发生位置: /src/parser/url_parser_tab.c	行号: 477
	步骤描述: 不使用返回值的函数'TOD_SAFE_FREE'应使用'void'说明.	
	代码片段:	

	<pre> 475: char *pass = pn ? strdup(p, pn) : NULL; 476: if (pn && !pass) { 477: TOD_SAFE_FREE(user); 478: return -1; 479: }</pre>	
	污染路径: 函数'TOD_SAFE_FREE'的定义点.	
125	缺陷发生位置: /testes/test_helper.c	行号: 147
	步骤描述: 不使用返回值的函数'W'应使用'void'说明.	
	代码片段: <pre> 145: if (r) { 146: char *t1 = cJSON_PrintUnformatted(json); 147: W("no item (%s) found in ==%s==", key, t1); 148: free(t1); 149: return -1;</pre>	
	污染路径: 函数'W'的定义点.	
126	缺陷发生位置: /samples/c/demo_topic.c	行号: 185
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: <pre> 183: sr = CALL_SQLConnect(hconn, (SQLCHAR*)dsn, SQL_NTS, (SQLCHAR*)uid, SQL_NTS, (SQLCHAR*)pwd, SQL_NTS); 184: if (FAILED(sr)) { 185: E("connect [dsn:%s,uid:%s,pwd:%s] failed", dsn, uid, pwd); 186: return -1; 187: }</pre>	
	污染路径: 函数'E'的定义点.	
127	缺陷发生位置: /testes/c/sqlite3_test.c	行号: 314
	步骤描述: 不使用返回值的函数'A'应使用'void'说明.	
	代码片段: <pre> 312: CALL_SQLFreeHandle(SQL_HANDLE_STMT, hstmt);</pre>	

	313: 314: A(r == 0, ""); 315: return (r FAILED(sr)) ? -1 : 0; 316: }	
	污染路径: 函数'A'的定义点.	
128	缺陷发生位置: /common/odbc_helpers.c	行号: 110
	步骤描述: 不使用返回值的函数'E'应使用'void'说明.	
	代码片段: 108: sr = CALL_SQLConnect(hconn, (SQLCHAR*)dsn, SQL_NTS, (SQLCHAR*)uid, SQL_NTS, (SQLCHAR*)pwd, SQL_NTS); 109: if (FAILED(sr)) { 110: E("connect [dsn:%s,uid:%s,pwd:%s] failed", dsn, uid, pwd); 111: return -1; 112: }	
	污染路径: 函数'E'的定义点.	

12 C/C++ 缺陷类型-代码质量-头文件问题

12.1 分项汇总表

序号	C/C++缺陷类型-代码质量-头文件问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4680	禁止在 #include 头文件前有可执行代码	4	严重
2	WK4676	禁止头文件重复包含	18	严重

12.2 缺陷/安全漏洞详解

12.2.1 严重类

12.2.1.1 WK4680 禁止在#include 头文件前有可执行代码

缺陷数量：4 个
缺陷描述： 禁止在#include 头文件前有可执行代码。
修复意见：

	步骤描述: 禁止在#include 头文件前有可执行代码
	代码片段: 32: #include <string.h> 33: 34: typedef struct _ejson_str_s _ejson_str_t; 35: typedef struct _ejson_kv_s _ejson_kv_t; 36: typedef struct _ejson_bin_s _ejson_bin_t;
	污染路径: 无

12.2.1.2 WK4676 禁止头文件重复包含

缺陷数量：18 个		
缺陷描述: 禁止头文件重复包含。		
修复意见: 无		
1	缺陷发生位置: /src/core/stmt.c	行号: 39
	步骤描述: 禁止头文件重复包含	
	代码片段: 37: #include "stmt.h" 38: #include "tables.h" 39: #include "taos_helpers.h" 40: #ifdef HAVE_TAOSWS /* { */ 41: #include "taosws_helpers.h"	
	污染路径: 原头文件定义点	
2	缺陷发生位置: /src/core/setup.c	行号: 27
	步骤描述: 禁止头文件重复包含	
	代码片段: 25: #include "internal.h" 26: 27: #include "os_port.h"	

	28: #include "conn.h" 29: #include "desc.h"	
	污染路径: 原头文件定义点	
3	缺陷发生位置: /src/parser/ts_parser_tab.c	行号: 29
	步骤描述: 禁止头文件重复包含	
	代码片段: 27: #include "../core/internal.h" // FIXME: 28: #include "log.h" 29: #include "parser.h" 30: #include "topic.h" 31: #include "var.h"	
	污染路径: 原头文件定义点	
4	缺陷发生位置: /src/core/topic.c	行号: 34
	步骤描述: 禁止头文件重复包含	
	代码片段: 32: #include "conn_parser.h" 33: #include "stmt.h" 34: #include "taos_helpers.h" 35: #ifdef HAVE_TAOSWS /* { */ 36: #include "taosws_helpers.h"	
	污染路径: 原头文件定义点	
5	缺陷发生位置: /src/core/primarykeys.c	行号: 33
	步骤描述: 禁止头文件重复包含	
	代码片段: 31: #include "errs.h" 32: #include "log.h" 33: #include "taos_helpers.h" 34: #include "tls.h" 35: #include "tsdb.h"	
	污染路径:	

	原头文件定义点	
6	缺陷发生位置: /src/core/tables.c	行号: 34
	步骤描述: 禁止头文件重复包含	
	代码片段: 32: #include "errs.h" 33: #include "log.h" 34: #include "taos_helpers.h" 35: #include "tls.h" 36: #include "tsdb.h"	
	污染路径: 原头文件定义点	
7	缺陷发生位置: /src/tests/basic.c	行号: 40
	步骤描述: 禁止头文件重复包含	
	代码片段: 38: #include "url_parser.h" 39: #include "tls.h" 40: #include "utils.h" 41: 42: #include <errno.h>	
	污染路径: 原头文件定义点	
8	缺陷发生位置: /src/core/typesinfo.c	行号: 31
	步骤描述: 禁止头文件重复包含	
	代码片段: 29: #include "errs.h" 30: #include "log.h" 31: #include "taos_helpers.h" 32: #include "tsdb.h" 33:	
	污染路径: 原头文件定义点	
9	缺陷发生位置: /tests/c/api_test.c	行号: 39
	步骤描述:	

	禁止头文件重复包含	
	代码片段: 37: #include <sqltext.h> 38: #include <stdarg.h> 39: #include <stddef.h> 40: #include <stdio.h> 41: #include <stdlib.h>	
	污染路径: 原头文件定义点	
10	缺陷发生位置: /src/core/conn.c	行号: 36
	步骤描述: 禁止头文件重复包含	
	代码片段: 34: #include "conn_parser.h" 35: #include "stmt.h" 36: #include "taos_helpers.h" 37: #ifdef HAVE_TAOSWS /* { */ 38: #include "taosws_helpers.h"	
	污染路径: 原头文件定义点	
11	缺陷发生位置: /src/parser/conn_parser_tab.c	行号: 28
	步骤描述: 禁止头文件重复包含	
	代码片段: 26: #include "conn.h" 27: #include "log.h" 28: #include "parser.h" 29: 30: #include "conn_parser.tab.h"	
	污染路径: 原头文件定义点	
12	缺陷发生位置: /src/parser/url_parser_tab.c	行号: 29
	步骤描述: 禁止头文件重复包含	
	代码片段: 27: #include "../core/internal.h" // FIXME: 28: #include "log.h"	

	29: #include "parser.h" 30: 31: #include <ctype.h>	
	污染路径: 原头文件定义点	
13	缺陷发生位置: /src/core/setup.c	行号: 30
	步骤描述: 禁止头文件重复包含	
	代码片段: 28: #include "conn.h" 29: #include "desc.h" 30: #include "enums.h" 31: #include "env.h" 32: #include "errs.h"	
	污染路径: 原头文件定义点	
14	缺陷发生位置: /src/parser/ext_parser_tab.c	行号: 30
	步骤描述: 禁止头文件重复包含	
	代码片段: 28: #include "insert_eval.h" 29: #include "log.h" 30: #include "parser.h" 31: #include "topic.h" 32: #include "var.h"	
	污染路径: 原头文件定义点	
15	缺陷发生位置: /src/core/env.c	行号: 31
	步骤描述: 禁止头文件重复包含	
	代码片段: 29: #include "errs.h" 30: #include "log.h" 31: #include "taos_helpers.h" 32: 33: static unsigned int _taos_init_failed = 0;	

	污染路径: 原头文件定义点	
16	缺陷发生位置: /src/core/tsdb.c	行号: 33
	步骤描述: 禁止头文件重复包含	
	代码片段: 31: #include "log.h" 32: #include "stmt.h" 33: #include "taos_helpers.h" 34: #ifdef HAVE_TAOSWS /* { */ 35: #include "taosws_helpers.h"	
	污染路径: 原头文件定义点	
17	缺陷发生位置: /src/parser/sqls_parser_tab.c	行号: 29
	步骤描述: 禁止头文件重复包含	
	代码片段: 27: #include "../core/internal.h" // FIXME: 28: #include "log.h" 29: #include "parser.h" 30: 31: #include "sqls_parser.tab.h"	
	污染路径: 原头文件定义点	
18	缺陷发生位置: /src/core/columns.c	行号: 33
	步骤描述: 禁止头文件重复包含	
	代码片段: 31: #include "log.h" 32: #include "tables.h" 33: #include "taos_helpers.h" 34: #include "tls.h" 35: #include "tsdb.h"	
	污染路径: 原头文件定义点	

13 C/C++缺陷类型-算术运算

13.1 分项汇总表

序号	C/C++缺陷类型-算术运算	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4707	禁止对逻辑表达式进行位运算	8	严重
2	WK4682	逻辑判别表达式中的运算项必须要使用括号	44	严重
3	WK5661	禁止对有符号类型进行移位运算	1	严重

13.2 缺陷/安全漏洞详解

13.2.1 严重类

13.2.1.1 WK4707 禁止对逻辑表达式进行位运算

缺陷数量：8 个		
缺陷描述： 禁止对逻辑表达式进行位运算。		
修复意见： 无		
1	缺陷发生位置： /src/tests/basic.c	行号： 574
步骤描述：		

	禁止对逻辑表达式进行位运算.	
	代码片段: 572: E("%s", buf); 573: } 574: if ((!!r) ^ (!match)) { 575: E("parsing @[%dL]:%s", __line__, s); 576: if (r) {	
	污染路径: 无	
2	缺陷发生位置:	行号:
	/tests/taos/taos_test.c	392
	步骤描述: 禁止对逻辑表达式进行位运算.	
	代码片段: 390: r = test_query_cjson(taos, sql, rs); 391: 392: r = !(r ^ !positive); 393: LOG_FINI(r, "%s case[%d] [sql:%s]", positive ? "positive" : "negative", icase+1, sql); 394:	
	污染路径: 无	
3	缺陷发生位置:	行号:
	/tests/c/odbc_test.c	574
	步骤描述: 禁止对逻辑表达式进行位运算.	
	代码片段: 572: r = test_query_cjson(hconn, sql, rs); 573: 574: r = !(r ^ !positive); 575: LOG_FINI(r, "%s case[%d] [sql:%s]", positive ? "positive" : "negative", icase+1, sql); 576:	
	污染路径: 无	
4	缺陷发生位置:	行号:
	/tests/taos/taos_test.c	1464
	步骤描述: 禁止对逻辑表达式进行位运算.	
	代码片段:	

	1462: r = run_case(json_case); 1463: 1464: r = !(r ^ !positive); 1465: LOG_FINI(r, "%s case[##d]", positive ? "positive" : "negative", i+1); 1466:	
	污染路径: 无	
5	缺陷发生位置: /testes/c/odbc_test.c	行号: 1330
	步骤描述: 禁止对逻辑表达式进行位运算.	
	代码片段: 1328: r = 0; 1329: } else { 1330: r = !(r ^ !positive); 1331: LOG_FINI(r, "%s case[##d]", positive ? "positive" : "negative", i+1); 1332: }	
	污染路径: 无	
6	缺陷发生位置: /testes/taos/taos_test.c	行号: 1341
	步骤描述: 禁止对逻辑表达式进行位运算.	
	代码片段: 1339: r = _run_executes(taos, sql, executes); 1340: 1341: r = !(r ^ !positive); 1342: LOG_FINI(r, "%s case[##d] [%s]", positive ? "positive" : "negative", icase+1, sql); 1343:	
	污染路径: 无	
7	缺陷发生位置: /src/testes/basic.c	行号: 50
	步骤描述: 禁止对逻辑表达式进行位运算.	
	代码片段: 48: {	

	49: int r = 0; 50: if ((!!s1) ^ (!!s2)) { 51: return s1 ? 1 : -1; 52: }	
	污染路径: 无	
8	缺陷发生位置: /tests/c/odbc_test.c	行号: 1180
	步骤描述: 禁止对逻辑表达式进行位运算.	
	代码片段: 1178: r = _run_executes(hconn, sql, executes); 1179: 1180: r = !(r ^ !positive); 1181: LOG_FINI(r, "%s case[##d] [%s]", positive ? "positive" : "negative", icense+1, sql); 1182:	
	污染路径: 无	

13.2.1.2 WK4682 逻辑判别表达式中的运算项必须要使用括号

缺陷数量：44 个		
缺陷描述: 逻辑判别表达式中的运算项必须要使用括号。		
修复意见: 无		
1	缺陷发生位置: /benchmark/tb.c	行号: 243
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 241: } 242: if (0 == strcasecmp(arg, "--port")) { 243: if (i+1 >= argc) { 244: DUMP("<port> expected after --port"); 245: return -1;	

	污染路径: 无	
2	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 553
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 551: n = snprintf(p, end - p, "t%d %s", i, s); 552: } 553: if (n < 0 n >= end - p) { 554: E("buffer too small"); 555: return -1;	
	污染路径: 无	
3	缺陷发生位置: /common/auxiliary.c	行号: 398
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 396: '0','1','2','3','4','5','6','7','8','9','a','b','c','d','e','f' 397: }; 398: if (nr / 2 <= n) { 399: errno = E2BIG; 400: return NULL;	
	污染路径: 无	
4	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1537
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1535: 1536: buf[0] = '\0'; 1537: if (lws_get_urlarg_by_name_safe(wsi, "password", buf, sizeof(buf)-1) >=0 && buf[0]) { 1538: session->password = strdup(buf); 1539: if (!session->password) {	
	污染路径: 无	
5	缺陷发生位置:	行号:

	/src/core/stmt.c	1908
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1906: snprintf(ix + i * 2, 3, "%02x", (const unsigned char)inbuf[i]); 1907: } 1908: for (size_t i=0; i<outbytes-outbytesleft && i<sizeof(ox)/2; ++i) { 1909: snprintf(ox + i * 2, 3, "%02x", (const unsigned char)outbuf[i]); 1910: }	
	污染路径: 无	
6	缺陷发生位置: /tests/c/edge_cases_test.c	行号: 270
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 268: } 269: 270: if (nr != batches * ARRAY_SIZE) { 271: D("internal logic error"); 272: r = -1;	
	污染路径: 无	
7	缺陷发生位置: /benchmark/taos_benchmark.c	行号: 719
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 717: 718: n = snprintf(p, end - p, "insert into benchmark_case0"); 719: if (n < 0 n >= end - p) { 720: E("buffer too small"); 721: return -1;	
	污染路径: 无	
8	缺陷发生位置:	行号:

	/src/tests/basic.c	1223
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1221: A(errno == E2BIG, "[%d]%", errno, strerror(errno)); 1222: A(inbytes - inbytesleft == 0, "0 expected, but got == %zd==", inbytes - inbytesleft); 1223: A(inbuf - utf8 == 0, ""); 1224: A(outbytes - outbytesleft == 0, ""); 1225: A(outbuf - buf == 0, "");	
	污染路径: 无	
9	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1507
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1505: 1506: buf[0] = '\0'; 1507: if (lws_get_urlarg_by_name_safe(wsi, "port", buf, sizeof(buf)-1) >= 0) { 1508: int bytes = 0; 1509: int port = 0;	
	污染路径: 无	
10	缺陷发生位置: /src/core/stmt.c	行号: 2242
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 2240: 2241: size_t n = ctx->end - ctx->curr; 2242: if (n > outbytesleft / 2 * 2) n = outbytesleft / 2 * 2; 2243: memcpy(outbuf, ctx->residual, n); 2244: outbuf += n;	
	污染路径: 无	
11	缺陷发生位置: /src/core/stmt.c	行号: 1905
	步骤描述:	

	逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1903: char ix[4096+1]; ix[0] = '\0'; 1904: char ox[4096+1]; ox[0] = '\0'; 1905: for (size_t i=0; i<inbytes && i<sizeof(ix)/2; ++i) { 1906: snprintf(ix + i * 2, 3, "%02x", (const unsigned char)inbuf[i]); 1907: }	
	污染路径: 无	
12	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1546
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1544: 1545: buf[0] = '\0'; 1546: if (lws_get_urlarg_by_name_safe(wsi, "db", buf, sizeof(buf)-1) >=0 && buf[0]) { 1547: session->db = strdup(buf); 1548: if (!session->db) {	
	污染路径: 无	
13	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1528
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1526: 1527: buf[0] = '\0'; 1528: if (lws_get_urlarg_by_name_safe(wsi, "user", buf, sizeof(buf)-1) >=0 && buf[0]) { 1529: session->user = strdup(buf); 1530: if (!session->user) {	
	污染路径: 无	
14	缺陷发生位置: /common/ejson_parser_tab.c	行号: 809
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	

	代码片段: 807: { 808: double epsilon = 1e-6; 809: if(fabs(l-r) < epsilon) return 0; 810: return (l<r) ? -1 : 1; 811: }	
	污染路径: 无	
15	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 535
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 533: 534: n = snprintf(p, end - p, "create table benchmark_case0 ("); 535: if (n < 0 n >= end - p) { 536: E("buffer too small"); 537: return -1;	
	污染路径: 无	
16	缺陷发生位置: /benchmark/tb.c	行号: 234
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 232: } 233: if (0 == strcasecmp(arg, "--db")) { 234: if (i+1 >= argc) { 235: DUMP("<db> expected after --db"); 236: return -1;	
	污染路径: 无	
17	缺陷发生位置: /benchmark/tb.c	行号: 207
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 205: } 206: if (0 == strcasecmp(arg, "--ip")) {	

	207: if (i+1 >= argc) { 208: DUMP("<ip> expected after --ip"); 209: return -1;	
	污染路径: 无	
18	缺陷发生位置: /src/tests/basic.c	行号: 1225
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1223: A(inbuf - utf8 == 0, ""); 1224: A(outbytes - outbytesleft == 0, ""); 1225: A(outbuf - buf == 0, ""); 1226: 1227: outbytes = 3;	
	污染路径: 无	
19	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 561
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 559: 560: n = snprintf(p, end - p, ""); 561: if (n < 0 n >= end - p) { 562: E("buffer too small"); 563: return -1;	
	污染路径: 无	
20	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1519
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1517: 1518: buf[0] = '\0'; 1519: if (lws_get_urlarg_by_name_safe(wsi, "ip", buf, sizeof(buf)-1) >= 0 && buf[0]) { 1520: session->ip = strdup(buf); 1521: if (!session->ip) {	

	污染路径: 无	
21	缺陷发生位置: /tests/c/sqlite3_test.c	行号: 303
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 301: } 302: 303: if (nr != batches * ARRAY_SIZE) { 304: D("internal logic error"); 305: r = -1;	
	污染路径: 无	
22	缺陷发生位置: /benchmark/taos_benchmark.c	行号: 700
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 698: n = snprintf(p, end - p, "t%zd %s", i, s); 699: } 700: if (n < 0 n >= end - p) { 701: E("buffer too small"); 702: return -1;	
	污染路径: 无	
23	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 584
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 582: n = snprintf(p, end - p, "t%zd", i); 583: } 584: if (n < 0 n >= end - p) { 585: E("buffer too small"); 586: return -1;	
	污染路径: 无	
24	缺陷发生位置:	行号:

	/common/auxiliary.c	385
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 383: return NULL; 384: } 385: if (n <= nr / 2) { 386: errno = E2BIG; 387: return NULL;	
	污染路径: 无	
25	缺陷发生位置: /src/core/conn.c	行号: 236
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 234: } while (0) 235: 236: _LOCAL_(statusInterval); 237: _LOCAL_(timezone); 238: _LOCAL_(locale);	
	污染路径: expanded from macro '_LOCAL_'	
26	缺陷发生位置: /src/core/stmt.c	行号: 3433
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 3431: long long v = strtoll(s, &end, 0); 3432: int e = errno; 3433: if (e == ERANGE && (v == LONG_MAX v == LONG_MIN)) { 3434: stmt_append_err_format(stmt, "22008", 0, 3435: "Datetime field overflow:timestamp is required, but got ==[%s]==", s);	
	污染路径: 无	
27	缺陷发生位置: /benchmark/tb.c	行号: 225
	步骤描述:	

	逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 223: } 224: if (0 == strcasecmp(arg, "--pass")) { 225: if (i+1 >= argc) { 226: DUMP("<pass> expected after --pass"); 227: return -1; </pre>	
	污染路径: 无	
28	缺陷发生位置:	行号:
	/benchmark/taos_benchmark.c	731
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 729: n = snprintf(p, end - p, "t%zd", i); 730: } 731: if (n < 0 n >= end - p) { 732: E("buffer too small"); 733: return -1; </pre>	
	污染路径: 无	
29	缺陷发生位置:	行号:
	/benchmark/taos_benchmark.c	708
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 706: 707: n = snprintf(p, end - p, ""); 708: if (n < 0 n >= end - p) { 709: E("buffer too small"); 710: return -1; </pre>	
	污染路径: 无	
30	缺陷发生位置:	行号:
	/common/auxiliary.c	415
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 413: { 414: size_t i = 0; </pre>	

	415: for (i=0; i+1 < len; i += 2) { 416: const char c1 = buf[i]; 417: const char c2 = buf[i+1];	
	污染路径: 无	
31	缺陷发生位置: /src/core/conn.c	行号: 239
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 237: _LOCAL_(timezone); 238: _LOCAL_(locale); 239: _LOCAL_(charset); 240: 241: #undef _LOCAL_	
	污染路径: expanded from macro '_LOCAL_'	
32	缺陷发生位置: /src/tests/basic.c	行号: 1056
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1054: }; 1055: 1056: for (size_t i=0; i<sizeof(cases)/sizeof(cases[0]); ++i) { 1057: string_t pattern = { 1058: .charset = charset,	
	污染路径: 无	
33	缺陷发生位置: /src/tests/basic.c	行号: 1224
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1222: A(inbytes - inbytesleft == 0, "0 expected, but got == %zd==", inbytes - inbytesleft); 1223: A(inbuf - utf8 == 0, ""); 1224: A(outbytes - outbytesleft == 0, ""); 1225: A(outbuf - buf == 0, ""); 1226:	

	污染路径: 无	
34	缺陷发生位置: /benchmark/taos_benchmark.c	行号: 682
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 680: 681: n = snprintf(p, end - p, "create table benchmark_case0"); 682: if (n < 0 n >= end - p) { 683: E("buffer too small"); 684: return -1;	
	污染路径: 无	
35	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 572
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 570: 571: n = snprintf(p, end - p, "insert into benchmark_case0"); 572: if (n < 0 n >= end - p) { 573: E("buffer too small"); 574: return -1;	
	污染路径: 无	
36	缺陷发生位置: /src/core/conn.c	行号: 238
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 236: _LOCAL_(statusInterval); 237: _LOCAL_(timezone); 238: _LOCAL_(locale); 239: _LOCAL_(charset); 240:	
	污染路径:	

	expanded from macro '_LOCAL_'	
37	缺陷发生位置: /src/core/conn.c	行号: 510
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 508: }; 509: int failed = 0; 510: for (size_t i=0; i<sizeof(convs)/sizeof(convs[0]); ++i) { 511: const char *from = convs[i].from; 512: const char *to = convs[i].to;	
	污染路径: 无	
38	缺陷发生位置: /src/core/stmt.c	行号: 4152
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 4150: } 4151: 4152: if (ll > UINT_MAX) { 4153: stmt_append_err_format(stmt, "22003", 0, 4154: "Numeric value out of range:conversion from `'%s` to `SQL_INTEGER` failed",	
	污染路径: 无	
39	缺陷发生位置: /tests/c/edge_cases_test.c	行号: 306
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 304: } 305: 306: if (nr != batches * ARRAY_SIZE) { 307: D("internal logic error"); 308: r = -1;	
	污染路径: 无	
40	缺陷发生位置:	行号:

	/tests/c/test_threads.c	385
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 383: 384: size_t count = arg->threads; 385: for (size_t i=0; i<count && i < sizeof(threads)/sizeof(threads[0]); ++i) { 386: r = pthread_create(threads + i, NULL, _routine_pthread_create, (void*)arg); 387: if (r) {	
	污染路径: 无	
41	缺陷发生位置: /src/core/conn.c	行号: 237
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 235: 236: _LOCAL_(statusInterval); 237: _LOCAL_(timezone); 238: _LOCAL_(locale); 239: _LOCAL_(charset);	
	污染路径: expanded from macro '_LOCAL_'	
42	缺陷发生位置: /src/tests/basic.c	行号: 1222
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1220: A(n == (size_t)-1, "-1 expected, but got ==%zd==", n); 1221: A(errno == E2BIG, "[%d]%", errno, strerror(errno)); 1222: A(inbytes - inbytesleft == 0, "0 expected, but got ==%zd==", inbytes - inbytesleft); 1223: A(inbuf - utf8 == 0, ""); 1224: A(outbytes - outbytesleft == 0, "");	
	污染路径: 无	
43	缺陷发生位置:	行号:

	/tests/c/sqlite3_test.c		267
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 265: } 266: 267: if (nr != batches * ARRAY_SIZE) { 268: D("internal logic error"); 269: r = -1;		
	污染路径: 无		
44	缺陷发生位置: /benchmark/tb.c		行号: 216
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 214: } 215: if (0 == strcasecmp(arg, "--user")) { 216: if (i+1 >= argc) { 217: DUMP("<user> expected after --user"); 218: return -1;		
	污染路径: 无		

13.2.1.3 WK5661 禁止对有符号类型进行移位运算

缺陷数量：1 个			
缺陷描述: 对有符号类型进行移位运算会导致不可预料的后果。			
修复意见: 无			
1	缺陷发生位置: /common/logger.c		行号: 163
	步骤描述: 禁止对有符号数'int(5)'进行二元位运算.		
	代码片段: 161: static void logger_to_syslog(const char *log) 162: {		

	163: int priority = LOG_SYSLOG LOG_USER LOG_INFO; 164: syslog(priority, "%s", log); 165: }
	污染路径: expanded from macro 'LOG_SYSLOG'

14 C/C++ 缺陷类型-其他内存相关问题

14.1 分项汇总表

序号	C/C++缺陷类型-其他内存相关问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4085	释放未动态分配的内存	7	严重
2	WK5388	释放并不在堆上的内存	14	严重

14.2 缺陷/安全漏洞详解

14.2.1 严重类

14.2.1.1 WK4085 释放未动态分配的内存

缺陷数量： 7 个
缺陷描述： 在指针还未指向动态分配的内存时,释放指针指向的内存。
修复意见： 无

1	缺陷发生位置： /tests/test_helper.c	行号： 112
	步骤描述： 禁止释放未动态分配的内存't1'.	
	代码片段： 110: char *t1 = cJSON_PrintUnformatted(json); 111: W("no item (%s) found in ==%s==", key, t1); 112: free(t1); 113: return NULL; 114: }	
	污染路径： 无	
2	缺陷发生位置： /tests/taos/taos_test.c	行号： 130
	步骤描述： 禁止释放未动态分配的内存'p'.	
	代码片段： 128: int r = cJSON_cmp_str(i_col, val, p); 129: 130: free(p); 131: 132: return r;	
	污染路径： 无	
3	缺陷发生位置： /tests/test_helper.c	行号： 127
	步骤描述： 禁止释放未动态分配的内存't2'.	
	代码片段： 125: W("item (%s) in ==%s== is not string ==%s==", t1, key, t2); 126: free(t1); 127: free(t2); 128: return NULL; 129: }	
	污染路径： 无	
4	缺陷发生位置： /tests/c/odbc_test.c	行号： 617
	步骤描述：	

	禁止释放未动态分配的内存'p1'.	
	代码片段: <pre> 615: if (!p1 !p2) { 616: E("out of memory"); 617: free(p1); free(p2); 618: return -1; 619: }</pre>	
	污染路径: 无	
5	缺陷发生位置: /tests/test_helper.c	行号: 126
	步骤描述: 禁止释放未动态分配的内存't1'.	
	代码片段: <pre> 124: char *t2 = cJSON_PrintUnformatted(val); 125: W("item (%s) in ==%s== is not string ==%s==", t1, key, t2); 126: free(t1); 127: free(t2); 128: return NULL;</pre>	
	污染路径: 无	
6	缺陷发生位置: /tests/test_helper.c	行号: 148
	步骤描述: 禁止释放未动态分配的内存't1'.	
	代码片段: <pre> 146: char *t1 = cJSON_PrintUnformatted(json); 147: W("no item (%s) found in ==%s==", key, t1); 148: free(t1); 149: return -1; 150: }</pre>	
	污染路径: 无	
7	缺陷发生位置: /tests/test_helper.c	行号: 119
	步骤描述: 禁止释放未动态分配的内存't1'.	
	代码片段: <pre> 117: char *t1 = cJSON_PrintUnformatted(json);</pre>	

	<pre> 118: W("no item (%s) found in ==%s==", key, t1); 119: free(t1); 120: return NULL; 121: }</pre>
	污染路径: 无

14.2.1.2 WK5388 释放并不在堆上的内存

缺陷数量： 14 个		
缺陷描述： 应用程序在指向未使用关联的堆分配函数（如 malloc（）、calloc（）或 realloc（））分配的内存的指针上调用 free（）。		
修复意见： 在释放指针之前，程序员应确保指针以前已在堆上分配，并且内存属于程序员。释放未分配的指针将导致程序中出现未定义的行为。		
1	缺陷发生位置：	行号：
	/taosws/server/taoswsd.c	584
	步骤描述： 禁止释放非堆内存'err'。	
	代码片段： <pre> 582: tws_err_t *err = lws_container_of(d, tws_err_t, node); 583: tws_err_release(err); 584: SAFE_FREE(err); 585: 586: return 0;</pre>	
	污染路径： expanded from macro 'SAFE_FREE'	
2	缺陷发生位置：	行号：
	/tests/taos/taos_test.c	130
	步骤描述： 禁止释放非堆内存'p'。	
	代码片段： <pre> 128: int r = cJSON_cmp_str(i_col, val, p); 129: 130: free(p); 131: 132: return r;</pre>	

	污染路径: 无	
3	缺陷发生位置: /taosws/server/taoswsd.c	行号: 550
	步骤描述: 禁止释放非堆内存'io'.	
	代码片段: 548: tws_io_t *io = lws_container_of(d, tws_io_t, node); 549: tws_session_release_io(session, io); 550: SAFE_FREE(io); 551: 552: return 0;	
	污染路径: expanded from macro 'SAFE_FREE'	
4	缺陷发生位置: /tests/test_helper.c	行号: 112
	步骤描述: 禁止释放非堆内存't1'.	
	代码片段: 110: char *t1 = cJSON_PrintUnformatted(json); 111: W("no item (%s) found in ==%s==", key, t1); 112: free(t1); 113: return NULL; 114: }	
	污染路径: 无	
5	缺陷发生位置: /src/utlis/utlis.c	行号: 1162
	步骤描述: 禁止释放非堆内存'node'.	
	代码片段: 1160: node->key = strdup(key); 1161: if (!node->key) { 1162: free(node); 1163: return -1; 1164: }	
	污染路径: 无	
6	缺陷发生位置:	行号:

	/tests/c/odbc_test.c		617
	步骤描述: 禁止释放非堆内存'p1'.		
	代码片段: 615: if (!p1 !p2) { 616: E("out of memory"); 617: free(p1); free(p2); 618: return -1; 619: }		
	污染路径: 无		
7	缺陷发生位置: /taosws/server/taoswsd.c		行号: 567
	步骤描述: 禁止释放非堆内存'step'.		
	代码片段: 565: lws_dll2_remove(d); 566: tws_json_serializer_step_t *step = lws_container_of(d, tws_json_serializer_step_t, node); 567: SAFE_FREE(step); 568: 569: return 0;		
	污染路径: expanded from macro 'SAFE_FREE'		
8	缺陷发生位置: /tests/test_helper.c		行号: 126
	步骤描述: 禁止释放非堆内存't1'.		
	代码片段: 124: char *t2 = cJSON_PrintUnformatted(val); 125: W("item (%s) in ==%s== is not string ==%s==", t1, key, t2); 126: free(t1); 127: free(t2); 128: return NULL;		
	污染路径: 无		
9	缺陷发生位置: /tests/test_helper.c		行号: 127
	步骤描述:		

	禁止释放非堆内存't2'.	
	代码片段: 125: W("item (%s) in ==%s== is not string ==%s==", t1, key, t2); 126: free(t1); 127: free(t2); 128: return NULL; 129: }	
	污染路径: 无	
10	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1615
	步骤描述: 禁止释放非堆内存'jio'.	
	代码片段: 1613: tws_jio_t *jio = lws_container_of(d, tws_jio_t, node); 1614: tws_jio_release(jio); 1615: SAFE_FREE(jio); 1616: 1617: return 0;	
	污染路径: expanded from macro 'SAFE_FREE'	
11	缺陷发生位置: /benchmark/odbc_benchmark.c	行号: 633
	步骤描述: 禁止释放非堆内存'data'.	
	代码片段: 631: SFREE(data[i]); 632: } 633: SFREE(data); 634: } 635:	
	污染路径: expanded from macro 'SFREE'	
12	缺陷发生位置: /tests/test_helper.c	行号: 148
	步骤描述: 禁止释放非堆内存't1'.	
	代码片段: 146: char *t1 = cJSON_PrintUnformatted(json);	

	147: W("no item (%s) found in ==%s==", key, t1); 148: free(t1); 149: return -1; 150: }	
	污染路径: 无	
13	缺陷发生位置: /testes/test_helper.c	行号: 119
	步骤描述: 禁止释放非堆内存't1'.	
	代码片段: 117: char *t1 = cJSON_PrintUnformatted(json); 118: W("no item (%s) found in ==%s==", key, t1); 119: free(t1); 120: return NULL; 121: }	
	污染路径: 无	
14	缺陷发生位置: /taosws/server/taoswsd.c	行号: 1572
	步骤描述: 禁止释放非堆内存'stmt'.	
	代码片段: 1570: lws_dll2_remove(d); 1571: ds_stmt_release(stmt); 1572: SAFE_FREE(stmt); 1573: 1574: return 0;	
	污染路径: expanded from macro 'SAFE_FREE'	